Local EIS Pipeline

Python technical reference

The Python concepts the pipeline is built from, how the modules are connected, and what every Python file is for.

Covers	
Package	local_eis_final.zip (folders core/, pipeline/, checks/, analysis/, plotting/, readers/, tests/, docs/)
Size	100 Python files (51 in tests/), about 43,135 lines including the two Databricks notebooks
Verified	Test suite run on the package: 453 passed, 3 skipped, 1 documented intentional failure
Companion	docs/LOCAL_EIS_TRAINING_GUIDE.md (training guide) and the package's own docs/PIPELINE_GUIDE.md (reference manual)
Date	October 2026

Contents

1. How to read this document . . . 6

2. The pipeline on one page . . . 6

2.1 Folder map . . . 7

2.2 Two pipelines . . . 7

3. How one run flows through the code . . . 8

3.1 The data objects handed between steps . . . 8

3.2 Who imports whom . . . 9

4. Python concepts used in the pipeline . . . 12

4.1 Language foundations . . . 12

Modules, imports and sys.path . . . 12

The if __name__ == "__main__" guard . . . 12

Functions, default and keyword arguments . . . 12

Type hints and from __future__ import annotations . . . 13

f-strings . . . 13

Comprehensions and generator expressions . . . 13

Lambda functions . . . 13

Closures (functions inside functions) . . . 14

Exceptions: raise, try/except, SystemExit . . . 14

Context managers (with) . . . 14

4.2 Object-oriented features . . . 14

Dataclasses . . . 15

Frozen dataclasses and replace() . . . 15

@property . . . 15

@classmethod alternative constructors . . . 16

Factory with __new__ . . . 16

Duck typing and monkey-patching . . . 16

Inheritance (used sparingly) . . . 17

4.3 Program structure and design patterns . . . 17

Medallion architecture (bronze / silver / gold) . . . 17

Single source of truth . . . 17

Module-level state and global . . . 18

Provenance: manifests, digests and the cache key . . . 18

Defensive build check . . . 18

Audit trail instead of silent deletion . . . 18

4.4 Python standard library . . . 19

pathlib.Path . . . 19

re (regular expressions) . . . 19

argparse . . . 19

json, csv . . . 19

logging . . . 20

hashlib, importlib, tempfile, shutil, time . . . 20
4.5 Scientific Python stack . . . 20
NumPy arrays and vectorisation . . . 20
Boolean masks, NaN handling . . . 20
Complex numbers . . . 21
Linear least squares (np.linalg) . . . 21
FFT and cross-correlation (np.fft) . . . 21
Memory-mapped files (np.memmap) . . . 21
SciPy optimisation . . . 22
pandas . . . 22
matplotlib and Plotly . . . 22
4.6 Testing with pytest . . . 22
Test functions, assert and conftest.py . . . 23
Fixtures: tmp_path and monkeypatch . . . 23
Parametrised tests . . . 23
Synthetic data with known truth . . . 23
4.7 Databricks-specific Python . . . 23
Notebook source format . . . 24
dbutils, spark, displayHTML, %run . . . 24
Unity Catalog Volumes and the workspace . . . 24
4.8 Concept → file index . . . 24
5. Libraries and what they are used for . . . 27
6. Every Python file . . . 29
6.1 (top level) . . . 29
Local EIS Pipeline Runner.py . . . 29
Local EIS Extra Analyses.py . . . 30
eis_paths.py . . . 31
run.py . . . 31
6.2 core/ . . . 32
core/config.py . . . 32
core/plate_conditions.py . . . 32
core/plate_stats.py . . . 33
core/r2d2_geometry.py . . . 34
core/utils.py . . . 35
6.3 pipeline/ . . . 36
pipeline/main.py . . . 36
pipeline/bronze.py . . . 37
pipeline/silver.py . . . 38
pipeline/gold.py . . . 40
pipeline/csv_pipeline.py . . . 41
pipeline/channel_lag.py . . . 42
pipeline/eis_local.py . . . 42
pipeline/eis_measurement_model.py . . . 44

pipeline/eis_validation.py . . . 44
pipeline/gamry_sync.py . . . 45
pipeline/hf_schedule.py . . . 46
pipeline/ladder_snap.py . . . 47
pipeline/neighbours.py . . . 47
pipeline/standard_eis.py . . . 48
pipeline/tone_estimation.py . . . 48
6.4 checks/ . . . 49
checks/card_diagnostics.py . . . 49
checks/card_gain.py . . . 50
checks/dc_closure.py . . . 51
checks/frequency_response.py . . . 51
checks/gamry_compare.py . . . 52
checks/hf_response.py . . . 53
checks/plausibility.py . . . 54
checks/segment_scale.py . . . 55
checks/validate_hf.py . . . 55
checks/via_resistance.py . . . 56
6.5 analysis/ . . . 56
analysis/ecm_drt.py . . . 56
analysis/polcurve.py . . . 57
6.6 plotting/ . . . 58
plotting/bench_plots.py . . . 58
plotting/figure_panels.py . . . 58
plotting/nyquist.py . . . 59
plotting/plate_figure.py . . . 59
plotting/plate_maps.py . . . 60
plotting/plate_plotly.py . . . 60
plotting/plate_style.py . . . 61
plotting/viewers.py . . . 62
6.7 readers/ . . . 63
readers/_vendor/zstd.py . . . 63
readers/abgleich.py . . . 63
readers/csv_source.py . . . 63
readers/gamry_dta.py . . . 65
6.8 docs/ . . . 66
docs/test_plans/make_test_plan.py . . . 66
6.9 tests/ . . . 66
tests/conftest.py . . . 66
tests/make_synth_famos.py . . . 66
tests/make_synth_sweep.py . . . 66
7. Commands you will use . . . 69
7.1 Rules for changing the code . . . 69

8. Glossary 69

1. How to read this document

This document is the **technical companion** to the training guide. It answers three questions:

• **Which Python concepts** does the pipeline use, what do they mean, where exactly are they used and why (Part 4).

• **How are the modules connected**: the run flow and the import graph (Parts 2 and 3).

• **What is every Python file for**: purpose, reason to exist, inputs and outputs, dependencies, the Python features it uses and an index of every top-level class and function (Part 6).

The function index, the dependency lists and the per-file concept tags are **generated from the source code** with Python's `ast` module, so they are complete and match the package exactly. The explanations were written by reading the code.

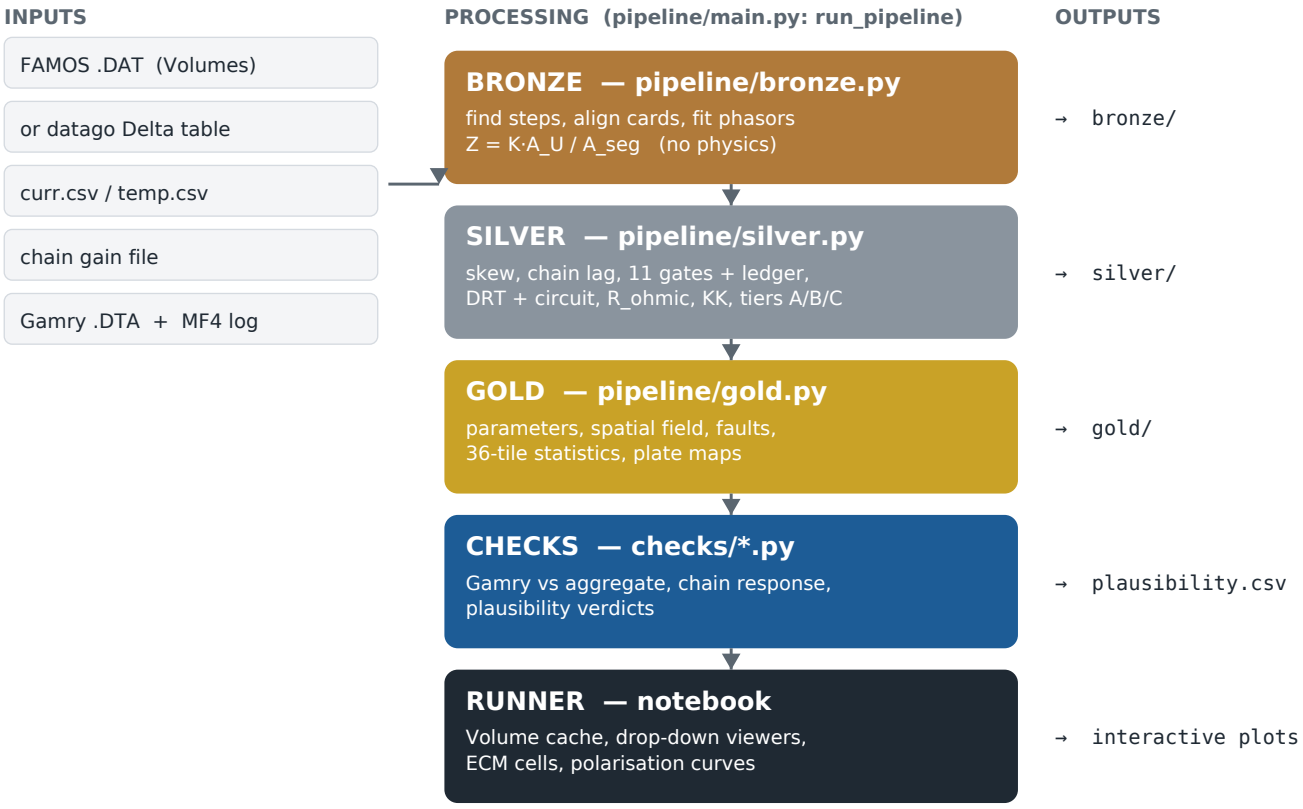
> Reading order for a newcomer: Part 2 (the picture) → Part 3 (the flow) → Part 4 (the concepts, with the code open beside you) → Part 6 for the files you are about to work on.

2. The pipeline on one page

For every one of the 72 segments of the R2-D2 measuring plate and for every excitation frequency, the software computes the local impedance

> **Z_seg(f) = K_seg(T) · A_U(f) / A_seg(f)** — A_U: cell-voltage phasor, A_seg: segment current-channel phasor, K: calibration constant at the segment temperature. Result in Ω·cm² (mΩ·cm² in the output files).

Most of the code exists to make that ratio trustworthy (right time windows, exact frequencies, timing between five separately-clocked cards, honest bookkeeping) and to turn 72 spectra into plate maps. The processing is organised in layers, each writing its own folder:



2.1 Folder map

Folder	Role	Files
(top level)	Entry points: the two Databricks notebooks, the sys.path helper and the command-line launcher.	4
core/	Shared foundations used by every other folder: the configuration, numerical utilities, the plate geometry, plate statistics and plate conditions.	5
pipeline/	The processing itself: the orchestrator (main.py), the three layers bronze, silver, gold, the separate CSV pipeline, and the signal-processing modules they call.	15
checks/	Diagnostics that judge whether a finished result is believable. None of them can stop a run.	10
analysis/	Optional scientific analyses on top of the results: a DRT-informed circuit fit and polarisation curves.	2
plotting/	Everything that draws: Nyquist/Bode figures, plate heat maps (static and interactive), the drop-down viewers and the common style.	8
readers/	File readers for everything that is not a FAMOS recording: Gamry .DTA, the Abgleich calibration files, CSV logger files, plus a compatibility shim.	4
tests/	About 450 pytest tests, the synthetic-data generators they use and a real logger fixture.	51
docs/	Documentation, plus one small script that generates a measurement test plan.	1

2.2 Two pipelines

`source_format = "famos"` (the normal case) runs bronze → silver → gold. `source_format = "csv"` runs `pipeline/csv_pipeline.py`, a separate pipeline for the single-clock CSV logger. Both share the geometry (`r2d2_geometry`), the calibration (`eis_local.PlateCalibration`), the physics formulas and, at the end, the checks.

3. How one run flows through the code

The numbered steps below are the actual call order when the notebook runs one condition. Function names are clickable in your editor: search for `def <name>`.

#	Where	What happens (functions called)
1	**Notebook: Setup cell**	import eis_paths; import + importlib.reload every module; build check (_BUILD_MARKERS).
2	**Notebook: Widgets cell**	dbutils.widgets → LEEPA, CONDITIONS, segments, run mode; fixed settings (BAND, MIN_SNR_DB, CHANNEL_LAG, GAMRY_SYNC …).
3	**Notebook: Datago cell**	datago available → bronze.FamosFile = DatagoFamosFile (monkey-patch); else read Volumes .DAT.
4	**Notebook: Run cell**	for each condition: cfg = DEFAULT.replace(...); cache hit → load; else main.run_pipeline(cfg).
5	**main.run_pipeline**	geom.use_plate(cfg.plate); cfg.save(config_used.json); csv → csv_pipeline.run_csv.
6	**bronze.run**	discover_files → inventory_channels (FamosFile) → PlateCalibration.load → estimate_card_lags → plate_temperatures → consensus_schedule (detect_schedule, ladder_snap, gamry_sync, hf_schedule) → refine_frequencies (tone_estimation) → card_gain.measure_reference → process_card per card (phasors_at, Z = K·A_ref/A_seg).
7	**silver.run**	dc_share_faults → fit_structural_skew per card → channel_lag.estimate → gamry_card_phase / card_hf_phase → process_segment per segment (gate_points, corrections, bayesian_drt, standard_eis, lin_kk, z_hit, tier) → dc_closure → cell_aggregate_full.
8	**gold.run**	collect_parameters (split_processes, re_at) → fit_field (Gaussian process) → label_faults → tile statistics → maps.
9	**main.whole_cell_and_plausibility**	gamry_compare.run → frequency_response.run → plausibility.check_run → plausibility.csv; write run_manifest.json.
10	**Notebook: result cells**	copy run folder to the Volume cache; PIPELINE_RESULTS[cond]; viewers.selector_html → displayHTML.

3.1 The data objects handed between steps

Every arrow between layers carries one of these dataclasses. Knowing them is the fastest way into the code.

Object	Defined in	Holds	Flows
Config	core/config.py	every setting of one run (frozen)	created by notebook/CLI, read by every stage
Step	pipeline/eis_local.py	one excitation step: freq, start, stop, amp, snr_db, thd, stationarity	bronze schedule
ChannelInfo / CardInfo	pipeline/bronze.py	one channel (card, ADC slot, kind) / one card (fs, n_ch, reference)	bronze inventory
BronzeSpectrum	pipeline/bronze.py	raw spectrum of one segment: freq, Z_raw, SNRs, THD, drift, samples per step, slot, K, T, u_dc	bronze → silver
BronzeRun	pipeline/bronze.py	schedule, channels, cards, spectra, lags, grid, digests	bronze.run → silver.run
SkewModel	pipeline/silver.py	per-card ADC skew: basis, dt0, k_slot, applied	silver
ChannelLag	pipeline/channel_lag.py	per-segment chain lag tau, residual, status	silver
SilverSpectrum	pipeline/silver.py	corrected + modelled spectrum, R_ohmic ± sd, R_pol, L, DRT, KK/Z-HIT, tier, flags	silver → gold
SilverRun	pipeline/silver.py	spectra, skew, DC closure, cell aggregate, rejection ledger	silver.run → gold.run, checks
SegmentRecord	pipeline/gold.py	one of 72 segments on the finished plate: class, tier, values, sd, fault	gold

Object	Defined in	Holds	Flows
GoldRun	pipeline/gold.py	records, spatial fields, statistics	gold.run → save, manifest
CsvMeasurement	readers/csv_source.py	whatever a CSV held, in one shape (time or frequency domain)	csv_source → csv_pipeline
GamrySweep / CellSweep	readers/gamry_dta.py / checks/gamry_compare.py	a Gamry .DTA sweep	chain gain, comparison
Check / Report	checks/plausibility.py	one verdict / all verdicts	plausibility.csv

3.2 Who imports whom

Generated from the `import` statements. "Imports" lists the pipeline's own modules a file uses; "imported by" lists the files that use it (tests excluded). Read it as the wiring diagram of the package.

Module	Imports (own modules)	Imported by
`analysis/ecm_drt.py`	—	Local EIS Extra Analyses, Local EIS Pipeline Runner
`analysis/polcurve.py`	config, eis_local, gamry_compare	Local EIS Pipeline Runner
`checks/card_diagnostics.py`	hf_response, nyquist	Local EIS Pipeline Runner
`checks/card_gain.py`	eis_local, r2d2_geometry, utils	Local EIS Pipeline Runner, bronze, dc_closure, frequency_response, main, plausibility, silver
`checks/dc_closure.py`	card_gain, gamry_compare, gamry_sync, r2d2_geometry, utils	Local EIS Pipeline Runner, main
`checks/frequency_response.py`	card_gain, gamry_dta, plausibility, utils	Local EIS Pipeline Runner, main
`checks/gamry_compare.py`	gamry_dta, utils	Local EIS Pipeline Runner, csv_pipeline, dc_closure, gamry_sync, main, nyquist, polcurve, silver
`checks/hf_response.py`	utils	Local EIS Pipeline Runner, card_diagnostics
`checks/plausibility.py`	card_gain, plate_stats, r2d2_geometry, silver, utils	Local EIS Pipeline Runner, frequency_response, main
`checks/segment_scale.py`	utils	Local EIS Pipeline Runner
`checks/validate_hf.py`	config, silver, utils	main
`checks/via_resistance.py`	—	Local EIS Pipeline Runner
`core/config.py`	—	Local EIS Extra Analyses, Local EIS Pipeline Runner, bronze, csv_pipeline, gold, main, plate_figure, plate_plotly, plate_style, polcurve, silver, validate_hf
`core/plate_conditions.py`	utils	main
`core/plate_stats.py`	r2d2_geometry	Local EIS Pipeline Runner, csv_pipeline, gold, plate_figure, plate_maps, plate_plotly, plausibility
`core/r2d2_geometry.py`	plate_style	Local EIS Pipeline Runner, bronze, card_gain, csv_pipeline, csv_source, dc_closure, eis_local, gold, main, neighbours, nyquist, plate_figure, plate_maps, plate_plotly, plate_stats, plausibility, silver, utils, viewers
`core/utils.py`	r2d2_geometry	Local EIS Pipeline Runner, bronze, card_gain, csv_pipeline, csv_source, dc_closure, frequency_response, gamry_compare, gold, hf_response, main, neighbours, plate_conditions, plausibility, segment_scale, silver, validate_hf
`eis_paths.py`	—	Local EIS Pipeline Runner, main, run

Module	Imports (own modules)	Imported by
pipeline/bronze.py	card_gain, config, eis_local, gamry_sync, hf_schedule, ladder_snap, r2d2_geometry, tone_estimation, utils	Local EIS Pipeline Runner, main, silver
pipeline/channel_lag.py	—	Local EIS Pipeline Runner, csv_pipeline, gamry_dta, silver
pipeline/csv_pipeline.py	channel_lag, config, csv_source, eis_local, eis_validation, gamry_compare, gold, nyquist, plate_figure, plate_stats, r2d2_geometry, silver, utils	Local EIS Pipeline Runner, main
pipeline/eis_local.py	eis_validation, hf_schedule, r2d2_geometry	Local EIS Pipeline Runner, abgleich, bronze, card_gain, csv_pipeline, gamry_dta, hf_schedule, polcurve
pipeline/eis_measurement_model.py	—	Local EIS Pipeline Runner, silver
pipeline/eis_validation.py	—	csv_pipeline, eis_local, silver
pipeline/gamry_sync.py	gamry_compare, hf_schedule	Local EIS Pipeline Runner, bronze, dc_closure
pipeline/gold.py	config, nyquist, plate_figure, plate_plotly, plate_stats, r2d2_geometry, silver, utils	Local EIS Pipeline Runner, csv_pipeline, main
pipeline/hf_schedule.py	eis_local	bronze, eis_local, gamry_sync
pipeline/ladder_snap.py	—	Local EIS Pipeline Runner, bronze
pipeline/main.py	bronze, card_gain, config, csv_pipeline, csv_source, dc_closure, eis_paths, frequency_response, gamry_compare, gold, plate_conditions, plausibility, r2d2_geometry, silver, utils, validate_hf	Local EIS Pipeline Runner
pipeline/neighbours.py	r2d2_geometry, utils	silver
pipeline/silver.py	bronze, card_gain, channel_lag, config, eis_measurement_model, eis_validation, gamry_compare, neighbours, r2d2_geometry, standard_eis, utils	Local EIS Pipeline Runner, csv_pipeline, gold, main, plausibility, validate_hf
pipeline/standard_eis.py	—	Local EIS Pipeline Runner, silver
pipeline/tone_estimation.py	—	Local EIS Pipeline Runner, bronze
plotting/bench_plots.py	—	Local EIS Pipeline Runner
plotting/figure_panels.py	—	Local EIS Pipeline Runner, viewers
plotting/nyquist.py	gamry_compare, plate_style, r2d2_geometry	Local EIS Pipeline Runner, card_diagnostics, csv_pipeline, gold
plotting/plate_figure.py	config, plate_stats, plate_style, r2d2_geometry	Local EIS Pipeline Runner, csv_pipeline, gold, viewers
plotting/plate_maps.py	plate_stats, plate_style, r2d2_geometry	Local EIS Pipeline Runner
plotting/plate_plotly.py	config, plate_stats, plate_style, r2d2_geometry	Local EIS Pipeline Runner, gold, viewers
plotting/plate_style.py	config	Local EIS Pipeline Runner, nyquist, plate_figure, plate_maps, plate_plotly, r2d2_geometry, viewers
plotting/viewers.py	figure_panels, plate_figure, plate_plotly, plate_style, r2d2_geometry	Local EIS Pipeline Runner
readers/_vendor/zstd.py	—	gamry_compare (via sys.path shim)
readers/abgleich.py	eis_local	Local EIS Pipeline Runner
readers/csv_source.py	gamry_dta, r2d2_geometry, utils	Local EIS Pipeline Runner, csv_pipeline, main
readers/gamry_dta.py	channel_lag, eis_local	Local EIS Extra Analyses, Local EIS Pipeline Runner, csv_source, frequency_response, gamry_compare

Module	Imports (own modules)	Imported by
`run.py`	eis_paths	—

The notebook `Local EIS Pipeline Runner.py` imports almost every module (39 of them) and is not repeated in the right-hand column.

> Layering rule visible in the table: `core/` imports nothing from the other folders except `plate_style` (for drawing the plate map); `pipeline/` uses `core/`, `readers/` and some `checks/`; `plotting/` only reads results. Keep it that way when adding code — a `core` module importing `silver` would create a cycle.

4. Python concepts used in the pipeline

Each concept is described with four parts: **what it is**, **where** it appears in this pipeline, **why** it was chosen there, and a **code excerpt** from the package (shortened where marked). Counts such as "34 files" are measured from the source with `ast`.

4.1 Language foundations

Modules, imports and sys.path

What it is. A .py file is a module. import silver runs it once and caches it in sys.modules. Python looks for modules in the folders listed in sys.path.

Where. eis_paths.py inserts core/, pipeline/, checks/, analysis/, plotting/ and readers/ into sys.path, so every module imports the others by plain name. run.py and the notebook use importlib.import_module and importlib.reload.

Why here. Databricks notebooks and the earlier flat layout import by plain name; this keeps both working without turning the code into an installed package. Caveat: two files with the same name in different folders would clash.

```
# eis_paths.py
ROOT = Path(__file__).resolve().parent
SUBDIRS = ("core", "pipeline", "checks", "analysis", "plotting", "readers")

def add(root=ROOT):
    for p in [root] + [root / d for d in SUBDIRS]:
        if str(p) not in sys.path:
            sys.path.insert(0, str(p))
add()                       # runs at import time

# run.py
mod = importlib.import_module(argv[0])     # name known only at runtime
return int(mod.main(argv[1:]) or 0)
```

The if __name__ == "__main__" guard

What it is. Code under this guard runs only when the file is executed directly, not when it is imported.

Where. 27 files: every module with a command-line use (main.py, gamry_dta.py, abgleich.py, the checks, the generators).

Why here. The same file is both an importable library and a command-line tool.

```
if __name__ == "__main__":
    raise SystemExit(main())
```

Functions, default and keyword arguments

What it is. Parameters can have defaults and be passed by name. Keyword-only arguments come after * in the signature.

Where. Nearly every function: run(cfg=DEFAULT, log=None), fit4(y, fs, f0, n_iter=12), cfg.replace(**kw).

Why here. Callers override only what they need; the defaults document the normal case.

```
def run_pipeline(cfg: Config, stop_after: str = "gold") -> dict: ...

def fit_spectrum(freq, Z, sigma_rel=None, rs_fixed="hf") -> dict: ...

cfg = DEFAULT.replace(condition="45A", f_min_hz=0.2)   # **kw
```

Type hints and from __future__ import annotations

What it is. Annotations describe expected types (cfg: Config, -> BronzeRun, dict[str, float], Path | None). They are documentation and are not enforced at runtime. The __future__ import makes annotations lazy strings.

Where. 86 files carry the __future__ import; 75 files annotate function signatures.

Why here. Annotations make the data flow between layers readable; lazy evaluation lets new syntax (X | None) work on older Pythons and lets classes refer to each other.

```
from __future__ import annotations

def process_card(fp: Path, cal: PlateCalibration, schedule: list[Step],
                 grid: dict, cfg: Config, log=None,
                 T_seg: dict[str, float] | None = None,
                 lag: int = 0) -> dict[str, BronzeSpectrum]: ...
```

f-strings

What it is. f"...{expr:format}..." builds a string with embedded, formatted expressions.

Where. 77 files; every log line.

Why here. Readable log messages with units and fixed decimals.

```
log.info(f"  R_ohmic {1000*r['mean']:.1f} +/- {1000*r['sd']:.1f} "
         f"mOhm cm2, spread {r['spread']:.2f}x")
```

Comprehensions and generator expressions

What it is. Compact syntax to build lists, dicts or sets, or to feed values lazily into functions like sum(), max(), any().

Where. 88 files use comprehensions, 63 use generator expressions.

Why here. Short, declarative transformations of segment dictionaries and file lists.

```
by_card = {}                                  # dict built in a loop …
j = {s: sp.j_dc for s, sp in spectra.items() if np.isfinite(sp.j_dc)}
I_meas = sum(j[s] * spectra[s].area_cm2 for s in j)   # generator expression
n_imp = sum(1 for s in out.values() if s.K_imputed)
```

Lambda functions

What it is. Small anonymous functions, usually passed as a key or callback.

Where. 46 files, mostly as sort keys: sorted(segments, key=int), max(cards, key=lambda c: ...).

Why here. Segment names are strings ('10' < '9' alphabetically), so sorting by int is essential.

```
for seg, s in sorted(sr.spectra.items(), key=lambda kv: int(kv[0])):
    ...
ref = max(fam.uc_names,
          key=lambda c: float(np.std(fam.channel(c)[::10])))
```

Closures (functions inside functions)

What it is. An inner function can read and modify objects of the function that encloses it.

Where. 34 files. The clearest case is the gate() helper in silver.gate_points, which records the first gate that rejects each point in the enclosing reason array.

Why here. Each gate stays one readable line, and the rejection ledger is filled as a side effect.

```
reason = np.array([""] * freq.size, dtype=object)

def gate(ok, name):
    """Apply a gate, recording it for the points it is the first to kill."""
    ok = np.asarray(ok, bool)
    newly = (reason == "") & ~ok
    reason[newly] = name
    return ok

keep  = gate(np.isfinite(freq) & (freq > 0), "not_finite")
keep &= gate((freq >= cfg.f_min_hz) & (freq <= f_top), "outside_band")
```

Exceptions: raise, try/except, SystemExit

What it is. raise signals an error; try/except catches it. raise SystemExit(msg) stops a program with a message.

Where. 40 files catch exceptions, 31 raise them. Three deliberate levels: fatal input errors raise SystemExit (no calibration, files ambiguous); a failing segment is caught and logged so the plate survives; every diagnostic is wrapped so it can never end a run.

Why here. “Refuse rather than guess” for inputs, “never lose the plate” for processing, “a diagnostic must not end a run” for checks.

```
if not cal.has_current_cal:
    raise SystemExit("bronze: --curr-cal is required. ...")

try:
    res = process_segment(sp, skew[sp.card], cfg, log, ledger=point_ledger)
except Exception as exc:                      # never lose the plate
    log.warning(f"    segment {seg}: {type(exc).__name__}: {exc}")
    res = None
```

Context managers (with)

What it is. with obj: ... runs setup and guaranteed cleanup around a block (files, temporary settings).

Where. 28 files: opening files, and np.errstate to silence expected divide-by-zero warnings locally.

Why here. Clean-up cannot be forgotten, and warning suppression stays local to the line that needs it.

```
with np.errstate(divide="ignore", invalid="ignore"):
    Y = Y + np.where(np.isfinite(z) & (z != 0), a / z, 0)

with open(path, "rb") as fh:
    head = fh.read(65536)
```

4.2 Object-oriented features

Dataclasses

What it is. @dataclass generates __init__, __repr__ and __eq__ from annotated class fields. field(default_factory=…) gives each instance its own mutable default; field(init=False) marks attributes computed later.

Where. 27 files, 44 classes: Config, Step, ChannelInfo, BronzeSpectrum, BronzeRun, SilverSpectrum, SilverRun, SegmentRecord, GoldRun, ChannelLag, CsvMeasurement, GamrySweep, Check, Report …

Why here. Every object handed between layers is a plain, typed record that prints readably and needs no boilerplate.

```
@dataclass
class Step:
    freq: float
    start: int
    stop: int
    amp: float
    snr_db: float
    thd: float
    stationarity: float

@dataclass
class BronzeRun:
    schedule: list[Step]
    spectra: dict[str, BronzeSpectrum]
    lags: dict = field(default_factory=dict)
```

Frozen dataclasses and replace()

What it is. frozen=True makes instances immutable. dataclasses.replace(obj, **changes) creates a modified copy.

Where. core/config.py: Config is frozen; Config.replace() wraps dataclasses.replace; preset() returns copies with gate settings changed.

Why here. A configuration that changes during a run is how an old notebook ended up with two different frequency limits in one figure. Immutability makes a run reproducible from config_used.json.

```
@dataclass(frozen=True)
class Config:
    plate: str = "gen1"
    f_min_hz: float = 0.15
    sigma_rel_max: float = 0.60
    ...
    def replace(self, **kw) -> "Config":
        return replace(self, **kw)

DEFAULT = Config()
cfg = DEFAULT.replace(condition="45A")    # DEFAULT itself is unchanged
cfg.f_min_hz = 1.0                        # -> FrozenInstanceError
```

@property

What it is. A method that is read like an attribute; the value is computed on access.

Where. 16 files: BronzeSpectrum.j_dc, SkewModel.slot_ratio, Plate.title, Segment.area_cm2, FamosFile.segment_names.

Why here. Derived values can never go out of sync with the data they come from.

```
@property
def segment_names(self) -> list[str]:
    return [n for n in self.names if re.fullmatch(r"\d+", n)]

@property
def area_cm2(self) -> float:
    return self.n_pads * PAD_AREA_CM2
```

@classmethod alternative constructors

What it is. A method that receives the class instead of an instance; used to build objects from other sources.

Where. Config.from_json / from_env / from_cli, PlateCalibration.load, Segment.from_pads / from_rect.

Why here. Several ways to create the same object, each with a clear name.

```
@classmethod
def from_json(cls, path) -> "Config":
    raw = json.loads(Path(path).read_text())
    return cls._coerce(raw)

cal = PlateCalibration.load(cfg.curr_cal, cfg.temp_cal)
```

Factory with __new__

What it is. __new__ creates the object before __init__ runs. Returning an instance of another class from it turns the class call into a factory.

Where. eis_local.FamosFile: FamosFile(path) looks at the file header and returns a FamosV1 (float32) or FamosV2 (float64) reader; if both fail it reports every attempt.

Why here. Callers write FamosFile(path) and never need to know which file dialect they got.

```
class FamosFile:
    def __new__(cls, path, *args, **kwargs):
        head = Path(path).open("rb").read(65536).decode("latin-1", "replace")
        looks_v2 = "|CN," in head
        order = [FamosV2, FamosV1] if looks_v2 else [FamosV1, FamosV2]
        why = []
        for reader in order:
            try:
                fam = reader(path)
            except Exception as exc:
                why.append(f"{reader.__name__}: {exc}"); continue
            if not _famos_plausible(fam):
                return fam
        raise ValueError("no FAMOS dialect could read this file: " + "; ".join(why))
```

Duck typing and monkey-patching

What it is. Python cares about what an object can do, not its class. Monkey-patching replaces a function or class in a module at runtime.

Where. The runner's DatagoFamosFile reads waveforms from a Spark Delta table but offers the same attributes (.names, .fs, .n_ch, .channel(), .position()) as the binary reader. The notebook assigns it to bronze.FamosFile, and bronze runs unchanged. Tests use pytest's monkeypatch the same way.

Why here. A new data source without touching the pipeline code.

```
# notebook, "Datago Source" cell (abridged)
class DatagoFamosFile:
    """FamosFile-compatible reader that pulls waveform data from datago."""
    def channel(self, name): ...           # -> np.ndarray from Spark rows
    def position(self, name): ...

_orig_FamosFile = _bronze_mod.FamosFile        # keep the original
_bronze_mod.FamosFile = _FamosFileDatagoShim   # datago first, file fallback
_bronze_mod.discover_files = _discover_datago_shim
```

Inheritance (used sparingly)

What it is. A class can extend another and override methods.

Where. core/utils.py: _SafeStreamHandler(logging.StreamHandler) overrides emit() so characters a Windows console cannot print (Ω, µ, °) do not produce logging tracebacks.

Why here. The code prefers plain data classes and functions; inheritance appears only where a library expects it.

```
class _SafeStreamHandler(logging.StreamHandler):
    def emit(self, record):
        msg = self.format(record)
        try:
            self.stream.write(msg + self.terminator)
        except UnicodeEncodeError:
            enc = getattr(self.stream, "encoding", None) or "ascii"
            self.stream.write(msg.encode(enc, "replace").decode(enc)
                              + self.terminator)
```

4.3 Program structure and design patterns

Medallion architecture (bronze / silver / gold)

What it is. Data moves through layers that each add value and write their own output: raw → corrected → business-level.

Where. pipeline/bronze.py, silver.py, gold.py; each has run() returning a dataclass and save() writing a folder. bronze.load() rebuilds a BronzeRun from its CSVs.

Why here. The slow raw read happens once; later layers can be rerun from saved files (--reevaluate) and every correction stays separate from measured data.

```
br = bronze.run(cfg, log);  bronze.save(br, cfg, log)    # -> out/bronze/
sr = silver.run(br, cfg, log); silver.save(sr, cfg, log)  # -> out/silver/
gr = gold.run(sr, cfg, log);  gold.save(gr, sr, cfg, log) # -> out/gold/
```

Single source of truth

What it is. Each fact is defined in exactly one place and read from there.

Where. Numbers: core/config.py. Areas: r2d2_geometry via utils.segment_areas(cfg). Calibration: eis_local.PlateCalibration. Map style: plotting/plate_style.py.

Why here. Earlier code had drifting copies of constants and could not be reproduced.

```
def segment_areas(cfg) -> dict[str, float]:
    """Precedence: --areas CSV > --equal-areas > true plate geometry."""
    if cfg.areas_file:
        return geom.areas_from_csv(cfg.areas_file)
    if cfg.equal_areas:
        return geom.equal_areas()
    return geom.areas()
```

Module-level state and global

What it is. Module variables are shared by everyone who imports the module; global inside a function rebinds them.

Where. r2d2_geometry.use_plate() rebinds SEGMENTS, LABEL_COL, LABEL_ROW and ACTIVE_PLATE. Code reads geom.SEGMENTS (attribute access) so it sees the switch.

Why here. One call selects the plate for the whole process. Pitfall: from r2d2_geometry import SEGMENTS would keep the old plate; always use geom.SEGMENTS, and call use_plate() before anything reads the geometry.

```
def use_plate(name):
    global ACTIVE_PLATE, SEGMENTS, LABEL_COL, LABEL_ROW
    p = plate(name)
    ACTIVE_PLATE, SEGMENTS = p, p.segments
    LABEL_COL, LABEL_ROW = p.label_col, p.label_row
    return p

plate = geom.use_plate(cfg.plate)     # first line of run_pipeline
```

Provenance: manifests, digests and the cache key

What it is. Every run records what produced it.

Where. config_used.json and run_manifest.json in every run folder; bronze stores a SHA-256 digest of the config and of the input files; the notebook's cache id is a hash of every result-changing setting (_CACHE_IDENTITY_KEYS).

Why here. Two runs with the same digest are the same computation; a changed setting gives a new cache entry instead of a stale one. Note: the code version is not in the hash, so after a code change use Run mode = rerun.

```
def _digest(items) -> str:
    h = hashlib.sha256()
    for it in items:
        h.update(str(it).encode("utf-8", "replace"))
    return h.hexdigest()[:16]

config_digest = _digest([json.dumps(cfg.to_dict(), sort_keys=True)])
```

Defensive build check

What it is. Verify at start-up that loaded modules contain the functions this version needs.

Where. Runner setup cell: _BUILD_MARKERS maps modules to required attribute names; any missing one stops the notebook with “These modules are not from this build”.

Why here. Importing a new zip over an old folder can mix versions and produce silently wrong maps.

```
_BUILD_MARKERS = {
    config: ['full_band'], bronze: ['to_own', 'to_common'],
    silver: ['r_ohmic_intercept', 'gamry_card_phase', 'standard_eis'], ...
}
for _m, _need in _BUILD_MARKERS.items():
    _missing = [a for a in _need if not hasattr(_m, a)]
```

Audit trail instead of silent deletion

What it is. Every discarded data point is recorded with a reason.

Where. silver: point_rejections.csv (one row per segment and point, with the gate that removed it) and segment_reach.csv (which gate limits each spectrum).

Why here. “Why does my spectrum stop at 400 Hz?” has an answer in a file instead of a guess.

```
ledger.append({"segment": sp.segment, "freq_hz": float(freq[i]),
               "kept": int(keep[i]), "reason": reason[i] or "unknown",
               "snr_db": snr[i], "cycles": cycles[i], "sigma_rel": s_all[i]})
```

4.4 Python standard library

pathlib.Path

What it is. Object-oriented file paths: path / "sub" / "file.csv", .exists(), .glob(), .read_text(), .stem, .stat().st_size.

Where. 49 files; every file access.

Why here. Portable between Windows, Linux and Databricks Volumes; no string concatenation of paths.

```
out = Path(cfg.out_dir) / "silver"
out.mkdir(parents=True, exist_ok=True)
files = sorted(Path(cfg.dat_dir).glob(pattern))
size = p.stat().st_size
```

re (regular expressions)

What it is. Pattern matching in text.

Where. 16 files: parsing FAMOS binary headers, deciding channel types, recognising condition names (45A vs 450A), CSV column names (s12, seg 12, u_s12 ...).

Why here. File headers and names follow patterns, not fixed positions.

```
m_cd = re.search(r"\|CD,\d+,([\d.eE+-]+)", raw)   # sample interval
fs = 1.0 / float(m_cd.group(1))
is_segment = re.fullmatch(r"\d+", name)            # channel "17" -> segment
_CURRENT_IN_NAME = re.compile(r"\d+(?:[.,]\d+)?\s*A(?![A-Za-z])")
```

argparse

What it is. Builds command-line interfaces with typed options, help and argument groups.

Where. 13 files: main.build_parser, Config.build_argparser, every check's main().

Why here. Each module can be run and tested from a terminal.

```
p = argparse.ArgumentParser(prog="main.py")
g = p.add_argument_group("hardware / source")
g.add_argument("--plate", choices=["gen1", "gen2"], default=None)
g.add_argument("--reevaluate", metavar="RUN_DIR")
a = p.parse_args(argv)
```

json, csv

What it is. Reading and writing JSON (manifests, configs) and CSV (all tables).

Where. json in 19 files, csv in 12; utils.write_table and utils.write_json wrap them and convert NumPy types.

Why here. Plain-text outputs that can be opened in Excel, pandas or a text editor.

```
utils.write_table(out / "segments_summary.csv", rows)     # list[dict] -> CSV
utils.write_json(out / "silver_manifest.json", {...})
cfg.save(out / "config_used.json")
```

logging

What it is. Leveled, configurable log output.

Where. utils.get_logger returns the shared "eis" logger with the console-safe handler; banner() and section() structure the run log.

Why here. One consistent, readable log; works on a Windows cp1252 console.

```
log = utils.get_logger(cfg.verbose)
utils.banner("SILVER  --  correction, modelling, validation", log)
log.warning(f"  segment {seg}: dropped -- {n} of {m} points survived")
```

hashlib, importlib, tempfile, shutil, time

What it is. Hashing; dynamic import/reload; temporary folders; copying folder trees; timing.

Where. hashlib in bronze and the notebook cache; importlib in run.py and the notebook; tempfile/shutil in the notebook (local run folder, copy to the Volume); time for elapsed time.

Why here. Infrastructure around the science.

```
importlib.reload(silver)                    # pick up an edited module
_TMP_BASE = Path(tempfile.gettempdir()) / f"eis_{os.getuid()}"
shutil.copytree(src_dir, dst_dir)           # copy a run folder
```

4.5 Scientific Python stack

NumPy arrays and vectorisation

What it is. ndarray holds numbers of one type; operations act on whole arrays at once (fast C loops) instead of Python loops.

Where. 85 files. Waveforms, spectra, masks, phasors.

Why here. Records hold millions of samples; Python loops would be far too slow.

```
t = np.arange(n) / fs
c, s = np.cos(w * t), np.sin(w * t)        # whole record at once
mag = np.abs(Z) * 1000.0                   # mOhm*cm2, every frequency
```

Boolean masks, NaN handling

What it is. Comparisons on arrays give boolean arrays used to select or combine; NaN marks missing values and np.isfinite / np.nan* functions skip them.

Where. 26 files use masks/np.where, 36 handle NaN. The 11 silver gates are combined boolean masks.

Why here. Missing or rejected points stay in place (as NaN or keep=False) so arrays keep their alignment with the frequency axis.

```
keep &= gate(np.isfinite(mag) & (mag > 0.5) & (mag < 800.0), "magnitude")
snr_med = float(np.nanmedian(snr[keep]))
Z = np.full(n_steps, np.nan, complex)       # filled step by step
```

Complex numbers

What it is. Python and NumPy support complex values (1j). A phasor and an impedance are complex numbers.

Where. 54 files. Z = K·A_ref/A_seg, de-skew Z·exp(j2πf·dt), Z.real, Z.imag, np.abs, np.angle, np.unwrap.

Why here. Impedance is inherently complex; complex arithmetic keeps magnitude and phase together.

```
Z[i] = K * A_ref / A_seg                        # complex ratio
def apply_delay(freq, Z, dt):
    """Undo an acquisition skew of dt seconds (all-pass)."""
    return np.asarray(Z) * np.exp(2j * np.pi * np.asarray(freq) * dt)
```

Linear least squares (np.linalg)

What it is. np.linalg.lstsq solves overdetermined linear systems; cholesky/solve are used for Gaussian processes.

Where. 14 files: sine fits, Lin-KK, Voigt models, the Bayesian DRT and the spatial field.

Why here. A sine at a known frequency is linear in its coefficients, so the best fit is one matrix solve.

```
D = np.column_stack([np.cos(w*t), np.sin(w*t), np.ones(n), t - t.mean()])
p, *_ = np.linalg.lstsq(D, y, rcond=None)
phasor = complex(p[0], -p[1])               # A = a - j b
```

FFT and cross-correlation (np.fft)

What it is. Fast Fourier transform; multiplying one spectrum by the conjugate of another gives a cross-correlation.

Where. 5 files: card alignment (bronze._best_lag), band-pass filtering, periodograms.

Why here. Correlating minutes of 25 kHz data directly would be far too slow; via the FFT it takes seconds.

```
m = 1 << int(np.ceil(np.log2(2 * n)))       # zero-pad to a power of 2
cc = np.fft.irfft(np.fft.rfft(a, m) * np.conj(np.fft.rfft(b, m)), m)
k = int(np.argmax(np.abs(cc)))              # lag of best agreement
```

Memory-mapped files (np.memmap)

What it is. Maps a binary file into memory without reading it; only the bytes actually accessed are loaded.

Where. eis_local.FamosV1/FamosV2.channel().

Why here. A card file is hundreds of MB; reading one channel should not load all 16.

```
mm = np.memmap(self.path, dtype="<f4", mode="r",       # little-endian float32
               offset=self.offset, shape=(self.n_samples, self.n_ch))
return np.asarray(mm[:, self.names.index(name)], dtype=np.float64)
```

SciPy optimisation

What it is. scipy.optimize.least_squares minimises a sum of squared residuals with bounds; minimize optimises general functions.

Where. 10 files: standard_eis, ecm_drt, csv_pipeline (circuit fits), silver (DRT hyper-parameters), tone_estimation.

Why here. Equivalent-circuit models are non-linear in their parameters; bounds keep them physical ($R \geq 0$, $0.5 \leq n \leq 1$).

```
s = np.abs(Z)                                  # modulus weighting
def resid(x):
    d = z_model(_to_p(x), f) - Z
    return np.concatenate([d.real / s, d.imag / s])

best = None
for x0 in starts:                               # multi-start
    r = least_squares(resid, x0, bounds=(lo, hi), max_nfev=4000,
                      x_scale="jac")
    if best is None or r.cost < best.cost:
        best = r
```

pandas

What it is. DataFrames: labelled tables with group-by, filtering and CSV I/O.

Where. 22 files: checks, plotting, notebook cells reading the output CSVs.

Why here. The output tables are analysed and plotted per condition, card or segment.

```
sp = pd.read_csv(run / "silver" / "spectra_clean.csv")
led = pd.read_csv(run / "silver" / "point_rejections.csv")
led[led.kept == 0].groupby("reason").size()
```

matplotlib and Plotly

What it is. matplotlib draws static figures (PNG), Plotly interactive ones (HTML/JS).

Where. matplotlib in 20 files (with the non-GUI "Agg" backend on servers); Plotly in 9 (go.Figure, make_subplots, figures embedded in viewers.selector_html).

Why here. PNGs for reports and the gold folder; interactive plots with hover and drop-downs in the notebook.

```
import matplotlib
matplotlib.use("Agg")                          # no screen on a cluster
fig.savefig(out / "nyquist.png", dpi=150, bbox_inches="tight")

html = viewers.selector_html({("45A", "Nyquist"): fig1,
                              ("45A", "|Z| Bode"): fig2},
                             dims=["Condition", "Plot"])
displayHTML(html)
```

4.6 Testing with pytest

Test functions, assert and conftest.py

What it is. pytest collects functions named test_* from files named test_*.py and checks plain assert statements. conftest.py runs first.

Where. tests/: 48 test files, about 457 tests. tests/conftest.py imports eis_paths.

Why here. Every behaviour that once broke has a test that plants the defect and checks it is caught.

```
def test_all_really_does_mean_all(tmp_path):
    folder = _campaign(tmp_path / "campaign",
                       "RO2611976-01_Current_{cond}_Test_01_Karte_{card}.DAT")
    cfg = DEFAULT.replace(dat_dir=folder, leepa="2611976", condition="ALL")
    assert len(bronze.discover_files(cfg)) == 20
```

Fixtures: tmp_path and monkeypatch

What it is. Fixtures are arguments pytest supplies automatically. tmp_path is a fresh temporary folder; monkeypatch replaces attributes for one test and restores them afterwards.

Where. tmp_path in most file-writing tests; monkeypatch in about 40 tests (e.g. replacing readers or console streams).

Why here. Tests never touch real data or leave files behind, and patched functions are always restored.

```
def test_x(tmp_path, monkeypatch):
    monkeypatch.setattr(bronze, "FamosFile", FakeReader)
    out = tmp_path / "run"
    ...
```

Parametrised tests

What it is. @pytest.mark.parametrize runs one test body over several inputs.

Where. 17 places, e.g. card alignment over several known shifts and bands.

Why here. One test covers positive, negative and zero cases.

```
@pytest.mark.parametrize("shift", [+12_345, -7_777, 0])
def test_a_known_shift_comes_back_with_the_right_sign(shift: int) -> None:
    ...
```

Synthetic data with known truth

What it is. Generate input data from a model whose answer is known, run the code, compare.

Where. tests/make_synth_famos.py, tests/make_synth_sweep.py, checks/validate_hf.py, csv_source._selftest.

Why here. Real measurements have no known truth; synthetic data with deliberately imposed timing faults proves the pipeline inverts the measurement rather than reproducing its own conventions.

```
def zarc(w, R, tau, n):
    return R / (1 + (1j * w * tau) ** n)

def true_z(params, f):
    w = 2 * np.pi * f
    return (params["R_ohm"] + 1j * w * params["L"]
            + zarc(w, params["R_ct"], params["tau_ct"], params["n_ct"])
            + zarc(w, params["R_mt"], params["tau_mt"], params["n_mt"]))
```

4.7 Databricks-specific Python

Notebook source format

What it is. A Databricks notebook can be stored as a .py file. Line 1 is "# Databricks notebook source"; cells are separated by "# COMMAND ----------"; "# DBTITLE 1,<title>" names a cell; "# MAGIC %md" lines are markdown.

Where. Local EIS Pipeline Runner.py and Local EIS Extra Analyses.py.

Why here. The notebooks are plain text, so they live in git and can be diffed and tested (tests/test_runner_*.py read their cells).

```
# Databricks notebook source
dbutils.library.restartPython()

# COMMAND ----------

# DBTITLE 1,Setup: sys.path + module imports
import sys
...
```

dbutils, spark, displayHTML, %run

What it is. Objects Databricks provides in every notebook: dbutils.widgets (input bar), spark (SQL on Delta tables), displayHTML (render HTML), %run (execute another notebook).

Where. Runner: widgets for order, conditions, segments, run mode; spark.sql against ps_xplatform_dev.rvadvtec_ops.datago_* tables; viewers via displayHTML. Extra Analyses: %run of the runner.

Why here. The notebook is the user interface; the modules stay plain Python and are testable outside Databricks.

```
dbutils.widgets.multiselect('conditions', 'ALL', ['ALL'] + CONDITIONS,
                            'Conditions')
LEEPA = dbutils.widgets.get('leepa_id')

rows = spark.sql(f"""
    SELECT DISTINCT m.orderId FROM {_META_TBL} m
    JOIN {_GP_TBL} gp ON m.measurement_id = gp.measurement_id
    WHERE gp.measurement_type = 'GALVEIS'""").collect()
```

Unity Catalog Volumes and the workspace

What it is. /Volumes/<catalog>/<schema>/<volume>/… is cloud storage mounted as a normal path; /Workspace/Users/… holds user files.

Where. Raw .DAT and Gamry folders, the result cache (EIS_Results) on Volumes; curr.csv/temp.csv/chain_gain under a personal workspace folder.

Why here. pathlib works on these paths like on a local disk. The personal paths (uum5fe@bosch.com) must be changed by a new owner.

```
FAMOS_ROOT = Path('/Volumes/ps_xplatform_dev/rvadvtec_dev/ev_rvadvtec_dev/Famos')
_CURR_CAL = Path('/Workspace/Users/uum5fe@bosch.com/curr.csv')
```

4.8 Concept → file index

Which files use which language feature, detected automatically from the syntax tree of each file (tests listed as a count).

Concept	Files (non-test)	Tests
comprehensions	Local EIS Extra Analyses.py, Local EIS Pipeline Runner.py, ecm_drt.py, polcurve.py, card_diagnostics.py, card_gain.py, dc_closure.py, frequency_response.py, gamry_compare.py, hf_response.py, plausibility.py, segment_scale.py, validate_hf.py, via_resistance.py, config.py, plate_conditions.py, plate_stats.py, r2d2_geometry.py, utils.py, make_test_plan.py, eis_paths.py, bronze.py, channel_lag.py, csv_pipeline.py, eis_local.py, eis_measurement_model.py, ... (+19)	43
__future__ annotations	ecm_drt.py, polcurve.py, card_diagnostics.py, card_gain.py, dc_closure.py, frequency_response.py, gamry_compare.py, hf_response.py, plausibility.py, segment_scale.py, validate_hf.py, via_resistance.py, config.py, plate_conditions.py, plate_stats.py, r2d2_geometry.py, utils.py, eis_paths.py, bronze.py, channel_lag.py, csv_pipeline.py, eis_local.py, eis_measurement_model.py, eis_validation.py, gamry_sync.py, gold.py, ... (+19)	41
f-strings	Local EIS Extra Analyses.py, Local EIS Pipeline Runner.py, ecm_drt.py, polcurve.py, card_diagnostics.py, card_gain.py, dc_closure.py, frequency_response.py, gamry_compare.py, hf_response.py, plausibility.py, segment_scale.py, validate_hf.py, via_resistance.py, config.py, plate_conditions.py, plate_stats.py, r2d2_geometry.py, utils.py, make_test_plan.py, bronze.py, channel_lag.py, csv_pipeline.py, eis_local.py, eis_measurement_model.py, eis_validation.py, ... (+21)	30
type hints	Local EIS Pipeline Runner.py, ecm_drt.py, polcurve.py, card_diagnostics.py, card_gain.py, dc_closure.py, frequency_response.py, gamry_compare.py, hf_response.py, plausibility.py, segment_scale.py, validate_hf.py, via_resistance.py, config.py, plate_conditions.py, plate_stats.py, r2d2_geometry.py, utils.py, eis_paths.py, bronze.py, channel_lag.py, csv_pipeline.py, eis_local.py, eis_measurement_model.py, eis_validation.py, gamry_sync.py, ... (+19)	30
generator expressions	Local EIS Extra Analyses.py, Local EIS Pipeline Runner.py, ecm_drt.py, polcurve.py, card_gain.py, dc_closure.py, frequency_response.py, gamry_compare.py, plausibility.py, segment_scale.py, validate_hf.py, config.py, plate_conditions.py, plate_stats.py, r2d2_geometry.py, bronze.py, channel_lag.py, csv_pipeline.py, eis_local.py, gamry_sync.py, gold.py, hf_schedule.py, ladder_snap.py, main.py, neighbours.py, silver.py, ... (+12)	25
complex numbers	Local EIS Extra Analyses.py, Local EIS Pipeline Runner.py, ecm_drt.py, card_diagnostics.py, card_gain.py, frequency_response.py, gamry_compare.py, hf_response.py, validate_hf.py, utils.py, bronze.py, channel_lag.py, csv_pipeline.py, eis_local.py, eis_measurement_model.py, eis_validation.py, gamry_sync.py, hf_schedule.py, ladder_snap.py, neighbours.py, silver.py, standard_eis.py, tone_estimation.py, nyquist.py, csv_source.py, gamry_dta.py	28
assert	—	49
pathlib	Local EIS Extra Analyses.py, Local EIS Pipeline Runner.py, polcurve.py, card_diagnostics.py, card_gain.py, dc_closure.py, frequency_response.py, gamry_compare.py, hf_response.py, plausibility.py, segment_scale.py, via_resistance.py, config.py, plate_stats.py, r2d2_geometry.py, utils.py, make_test_plan.py, eis_paths.py, bronze.py, csv_pipeline.py, eis_local.py, gamry_sync.py, gold.py, main.py, silver.py, nyquist.py, ... (+5)	18
pytest tests	—	48
lambda	Local EIS Extra Analyses.py, Local EIS Pipeline Runner.py, ecm_drt.py, polcurve.py, card_diagnostics.py, card_gain.py, frequency_response.py, gamry_compare.py, hf_response.py, plausibility.py, segment_scale.py, via_resistance.py, config.py, plate_conditions.py, r2d2_geometry.py, utils.py, bronze.py, channel_lag.py, csv_pipeline.py, eis_local.py, gamry_sync.py, gold.py, hf_schedule.py, ladder_snap.py, neighbours.py, silver.py, ... (+6)	14
try/except	Local EIS Extra Analyses.py, Local EIS Pipeline Runner.py, ecm_drt.py, polcurve.py, card_diagnostics.py, card_gain.py, dc_closure.py, frequency_response.py, gamry_compare.py, hf_response.py, plausibility.py, segment_scale.py, via_resistance.py, plate_stats.py, r2d2_geometry.py, utils.py, bronze.py, channel_lag.py, csv_pipeline.py, eis_local.py, gamry_sync.py, gold.py, ladder_snap.py, main.py, neighbours.py, silver.py, ... (+12)	2
classes	Local EIS Pipeline Runner.py, polcurve.py, card_gain.py, gamry_compare.py, plausibility.py, config.py, plate_conditions.py, r2d2_geometry.py, utils.py, bronze.py, channel_lag.py, csv_pipeline.py, eis_local.py, gamry_sync.py, gold.py, hf_schedule.py, neighbours.py, silver.py, tone_estimation.py, bench_plots.py, plate_plotly.py, abgleich.py, csv_source.py, gamry_dta.py	13

Concept	Files (non-test)	Tests
NaN handling	Local EIS Extra Analyses.py, Local EIS Pipeline Runner.py, ecm_drt.py, polcurve.py, card_diagnostics.py, card_gain.py, dc_closure.py, frequency_response.py, gamry_compare.py, plausibility.py, validate_hf.py, plate_conditions.py, plate_stats.py, utils.py, bronze.py, channel_lag.py, csv_pipeline.py, eis_local.py, eis_measurement_model.py, eis_validation.py, gold.py, hf_schedule.py, ladder_snap.py, neighbours.py, silver.py, standard_eis.py, ... (+5)	5
pytest	—	35
closures / nested functions	Local EIS Extra Analyses.py, Local EIS Pipeline Runner.py, ecm_drt.py, card_gain.py, gamry_compare.py, plausibility.py, plate_conditions.py, r2d2_geometry.py, utils.py, bronze.py, channel_lag.py, csv_pipeline.py, eis_measurement_model.py, gamry_sync.py, gold.py, hf_schedule.py, ladder_snap.py, silver.py, standard_eis.py, tone_estimation.py, bench_plots.py, figure_panels.py, plate_figure.py, plate_plotly.py, viewers.py, csv_source.py	8
pytest fixtures	—	33
raising exceptions	Local EIS Pipeline Runner.py, polcurve.py, card_diagnostics.py, dc_closure.py, frequency_response.py, hf_response.py, plausibility.py, segment_scale.py, validate_hf.py, via_resistance.py, config.py, plate_stats.py, r2d2_geometry.py, utils.py, bronze.py, csv_pipeline.py, eis_local.py, main.py, neighbours.py, tone_estimation.py, plate_figure.py, viewers.py, abgleich.py, csv_source.py, gamry_dta.py, run.py	5
context managers (with)	Local EIS Pipeline Runner.py, card_gain.py, dc_closure.py, frequency_response.py, gamry_compare.py, hf_response.py, plausibility.py, segment_scale.py, r2d2_geometry.py, utils.py, make_test_plan.py, bronze.py, csv_pipeline.py, eis_local.py, eis_measurement_model.py, hf_schedule.py, neighbours.py, silver.py, tone_estimation.py, abgleich.py, csv_source.py, gamry_dta.py	6
dataclasses	Local EIS Pipeline Runner.py, polcurve.py, card_gain.py, gamry_compare.py, plausibility.py, config.py, plate_conditions.py, r2d2_geometry.py, utils.py, bronze.py, channel_lag.py, csv_pipeline.py, eis_local.py, gamry_sync.py, gold.py, hf_schedule.py, neighbours.py, silver.py, tone_estimation.py, bench_plots.py, plate_plotly.py, abgleich.py, csv_source.py, gamry_dta.py	3
if __name__ == "__main__"	card_diagnostics.py, dc_closure.py, frequency_response.py, hf_response.py, plausibility.py, segment_scale.py, validate_hf.py, via_resistance.py, config.py, r2d2_geometry.py, utils.py, bronze.py, csv_pipeline.py, eis_local.py, eis_measurement_model.py, eis_validation.py, main.py, neighbours.py, tone_estimation.py, plate_figure.py, abgleich.py, csv_source.py, gamry_dta.py, run.py	3
boolean masks / np.where	Local EIS Extra Analyses.py, ecm_drt.py, card_diagnostics.py, card_gain.py, hf_response.py, segment_scale.py, utils.py, bronze.py, csv_pipeline.py, eis_local.py, eis_measurement_model.py, gold.py, ladder_snap.py, neighbours.py, silver.py, tone_estimation.py, csv_source.py	9
pandas	Local EIS Extra Analyses.py, Local EIS Pipeline Runner.py, polcurve.py, card_diagnostics.py, frequency_response.py, hf_response.py, segment_scale.py, via_resistance.py, plate_stats.py, nyquist.py, viewers.py	11
matplotlib	Local EIS Extra Analyses.py, Local EIS Pipeline Runner.py, polcurve.py, card_diagnostics.py, dc_closure.py, frequency_response.py, gamry_compare.py, via_resistance.py, r2d2_geometry.py, csv_pipeline.py, eis_local.py, gold.py, nyquist.py, plate_figure.py, plate_maps.py, plate_style.py	4
JSON	Local EIS Pipeline Runner.py, polcurve.py, dc_closure.py, frequency_response.py, hf_response.py, segment_scale.py, via_resistance.py, config.py, utils.py, bronze.py, csv_pipeline.py, eis_local.py, main.py, viewers.py, abgleich.py, csv_source.py	3
@property	card_gain.py, gamry_compare.py, plausibility.py, plate_conditions.py, r2d2_geometry.py, utils.py, bronze.py, eis_local.py, hf_schedule.py, neighbours.py, silver.py, bench_plots.py, abgleich.py, csv_source.py, gamry_dta.py	1
regular expressions	Local EIS Extra Analyses.py, polcurve.py, card_diagnostics.py, card_gain.py, gamry_compare.py, via_resistance.py, utils.py, bronze.py, eis_local.py, gamry_sync.py, figure_panels.py, abgleich.py, csv_source.py, gamry_dta.py	2

Concept	Files (non-test)	Tests
linear algebra (np.linalg)	Local EIS Extra Analyses.py, utils.py, bronze.py, csv_pipeline.py, eis_local.py, eis_measurement_model.py, eis_validation.py, gold.py, hf_schedule.py, silver.py, standard_eis.py, tone_estimation.py, abgleich.py, csv_source.py	0
argparse CLI	card_diagnostics.py, dc_closure.py, frequency_response.py, hf_response.py, plausibility.py, segment_scale.py, via_resistance.py, config.py, csv_pipeline.py, eis_local.py, main.py, abgleich.py, gamry_dta.py	0
csv module	Local EIS Pipeline Runner.py, polcurve.py, card_gain.py, dc_closure.py, gamry_compare.py, plausibility.py, make_test_plan.py, bronze.py, plate_figure.py	3
np.errstate	card_gain.py, frequency_response.py, utils.py, bronze.py, csv_pipeline.py, eis_local.py, eis_measurement_model.py, hf_schedule.py, neighbours.py, silver.py, tone_estimation.py, csv_source.py	0
SciPy	Local EIS Extra Analyses.py, ecm_drt.py, bronze.py, channel_lag.py, csv_pipeline.py, silver.py, standard_eis.py, tone_estimation.py, plate_figure.py	1
plotly	Local EIS Extra Analyses.py, Local EIS Pipeline Runner.py, bench_plots.py, figure_panels.py, nyquist.py, plate_plotly.py	3
sys.path	Local EIS Pipeline Runner.py, gamry_compare.py, eis_paths.py, main.py	3
frozen dataclass	gamry_compare.py, plausibility.py, config.py, r2d2_geometry.py, abgleich.py, gamry_dta.py	0
FFT (np.fft)	bronze.py, csv_pipeline.py, eis_local.py, hf_schedule.py, tone_estimation.py	0
importlib	Local EIS Extra Analyses.py, Local EIS Pipeline Runner.py, gamry_compare.py, run.py	1
logging	utils.py	3
@classmethod	config.py, r2d2_geometry.py, eis_local.py	0
__new__ factory	Local EIS Pipeline Runner.py, eis_local.py	0
global state	r2d2_geometry.py, utils.py	0
inheritance	utils.py	1
Databricks dbutils	Local EIS Pipeline Runner.py	0
Spark SQL	Local EIS Pipeline Runner.py	0
__post_init__	eis_local.py	0
generators (yield)	utils.py	0
hashing	bronze.py	0
module __getattr__	plate_figure.py	0
np.memmap	eis_local.py	0

5. Libraries and what they are used for

Library	Used for	Where
numpy	arrays, complex arithmetic, linear algebra, FFT, memmap	almost every module
scipy	optimize.least_squares / minimize (circuit and DRT fits)	standard_eis, ecm_drt, csv_pipeline, silver, tone_estimation
pandas	reading and analysing the output CSVs	checks, plotting, notebook
matplotlib	static figures (PNG maps, Nyquist, checks)	gold, plotting, checks, r2d2_geometry
plotly	interactive figures and viewers	plotting, notebook
asammdf (+ zstandard)	reading the MF4 bench log	gamry_compare, bench_plots
pytest	test framework	tests/
IPython	display() of images/HTML in notebooks	notebook cells
pathlib	file paths	49 files
dataclasses	data containers, the Config	27 files

Library	Used for	Where
re	header and name parsing	16 files
json / csv	manifests, configs, tables	~30 files
argparse	command-line interfaces	13 files
logging	run log	utils (+ users)
hashlib	config/input digests, cache id	bronze, notebook
importlib	dynamic import and reload	run.py, notebook
tempfile / shutil / os	local run folders, copy to cache	notebook, tests

Install for local work: `pip install numpy scipy matplotlib pandas plotly pytest` (+ `asammdf zstandard` for MF4 logs). Databricks ML runtimes already contain the scientific packages; `requirements.txt` is unpinned on purpose.

6. Every Python file

For each file: **purpose**, **why it exists**, **inputs → outputs**, its connections, the Python features detected in it, and an index of all top-level classes and functions with the first line of their docstring. Test files are summarised in one table at the end (6.9).

6.1 (top level)

Entry points: the two Databricks notebooks, the sys.path helper and the command-line launcher.

Local EIS Pipeline Runner.py

3,649 lines · 1 classes · 39 functions

Purpose. The main Databricks notebook and the only file a normal user runs. It finds its own folder, imports and reloads every module, checks that all modules come from the same build, creates the widget bar (order, conditions, segments, run mode), builds a Config per condition, calls main.run_pipeline, stores the result in the Volume cache and then shows every result as one interactive plot with drop-downs.

Why it exists. Notebook users should not need to touch Python modules. All site-specific paths, fixed evaluation settings, the cache logic and the display code live here; the science lives in the modules.

Inputs → outputs. Widgets + Volumes/datago data + calibration files → run folders on the Volume, PIPELINE_RESULTS dict, interactive figures.

Depends on	abgleich, bench_plots, bronze, card_diagnostics, card_gain, channel_lag, config, csv_pipeline, csv_source, dc_closure, ecm_drt, eis_local, eis_measurement_model, eis_paths, figure_panels, frequency_response, gamry_compare, gamry_dta, gamry_sync, gold, hf_response, ladder_snap, main, nyquist, plate_figure, plate_maps, plate_plotly, plate_stats, plate_style, plausibility, polcurve, r2d2_geometry, segment_scale, silver, standard_eis, tone_estimation, utils, via_resistance, viewers
Used by	—
Libraries	IPython, asammdf, csv, dataclasses, gc, hashlib, importlib, inspect, json, matplotlib, numpy, os, pandas, pathlib, plotly, re, shutil, subprocess, sys, tempfile, time
Python features	Databricks dbutils, JSON, NaN handling, Spark SQL, __new__ factory, classes, closures / nested functions, complex numbers, comprehensions, context managers (with), csv module, dataclasses, f-strings, generator expressions, importlib, lambda, matplotlib, pandas, pathlib, plotly, raising exceptions, sys.path, try/except, type hints

Class / function	What it does (first docstring line)
`class DatagoFamosFile`	FamosFile-compatible reader that pulls waveform data from datago. Methods: _load_metadata, position, channel, _load_channel, load_all_channels.
`_find_pipeline_dir()`	—
`show_by_condition(figs, title, note, save)`	{condition: figure} -> ONE plot with Condition (and Plot) drop-downs.
`viewer_dirs()`	Where the selector viewers are saved: the run folder of this plate on the results Volume (when cache writing is on) and the local run folder.
`save_viewer(html, name, title)`	Save a viewer exactly as shown (drop-downs and all) as <name>.html.
`skip_for_csv(what)`	True (and says so) when the selected format is CSV: the cell needs FAMOS .DAT files, an order ID, the Abgleich or the Gamry sweeps, which a CSV measurement does not have. Use as the first line of such a cell:
`show_fig(fig, html)`	Show a (possibly multi-panel) Plotly figure, one panel at a time.
`_w(name, default)`	—
`condition_sort_key(cond)`	'45A' < '60A' < '150A' < '450A': by the number, then by the text.
`parse_conditions(raw, available)`	The multi-select value -> the conditions to evaluate.
`gamry_search_roots(leepa)`	—
`plate_version(leepa, quiet)`	The build token for this order, e.g. 'V26_092', or None.

Class / function	What it does (first docstring line)
`gamry_root_for(leepa)`	First folder from the search roots that holds any .dta, or None.
`gamry_files(leepa, version)`	The .dta sweeps that belong to this order. Never another campaign's.
`gamry_spectra(leepa, quiet)`	{condition: DataFrame(freq_hz, z_re_mohm_cm2, z_im_mohm_cm2)} -- the order's whole-cell Gamry sweeps, EVERY point exactly as in the file (nyquist.gamry_frame): no band limit, no interpolation, no inductance removed. From
`_widget()`	First widget that exists, else a notebook global, else the default.
`hf_response_file(leepa)`	The response file the runs of this order apply (HF_RESPONSE), or None.
`_bode_dir()`	The Abgleich bode/ folder, when the "Chain response" cell has one.
`_seg_list(name)`	—
`discover_famos_file_ids(leepa)`	Find FAMOS card recordings in datago for a Leepa.
`_run_identity(mode, f_min, f_max, snr)`	Short digest of the settings that decide what a run keeps.
`_cache_dir(leepa, cond, snr, mode)`	—
`_cache_exists(leepa, cond, snr)`	True if usable cached results exist on the Volume.
`_save_to_cache(out_dir, leepa, cond, snr)`	Copy pipeline output to Volume for persistence across sessions.
`_load_from_cache(leepa, cond, snr)`	Return a PIPELINE_RESULTS-compatible dict pointing to the cached dir.
`cache_plan(leepa, cond, snr, mode)`	What happens to this condition: ('load'|'run'|'skip', why).
`_entry_info(cd)`	Describe one cache directory without trusting it to be complete.
`_cache_conditions(leepa)`	Every (condition, snr_tag, dir) cached under this Leepa.
`selected_conditions()`	The conditions the widgets asked for -- NOT whatever is on the Volume.
`result_dir(leepa, cond, snr, mode, prefer_live)`	(directory, provenance) for one condition, newest evidence first.
`_legacy_snr_names(snr)`	The snr_<value> spellings the old cache layout used.
`describe_source(cond, d, prov)`	One line naming what is about to be plotted, and how old it is.
`_esc(x)`	—
`_fmt(v, spec, dash)`	—
`_age_cell(days)`	Age, coloured. Stale cache is the failure mode this panel exists for.
`spectra_csv(out_dir)`	Where this run's per-segment spectra ended up.
`maps_dir(out_dir)`	Where the plate maps and summary tables ended up.
`_p_vector(res)`	params dict -> the flat vector z_model expects.
`_fit_one(f_hz, Z_mohm, sigma_rel)`	Fit one spectrum given in mOhm.cm2. Returns everything the plots need.
`_grade(v, lo, hi)`	Pick a colour from the sequential ramp for value v.

Local EIS Extra Analyses.py

1,356 lines · 0 classes · 13 functions

Purpose. A second notebook with optional diagnostics: plate numbering check, Gamry-vs-local decomposition, Lin-KK, Tikhonov DRT, the DRT-informed equivalent-circuit fit and a comparison of two ECM methods. Its first cell executes the main notebook with %run, so all variables are available.

Why it exists. Keeps the main notebook short and result-focused; deeper analyses are opt-in.

Inputs → outputs. PIPELINE_RESULTS from the runner → extra figures and tables.

Depends on	config, ecm_drt, gamry_dta
Used by	—
Libraries	IPython, importlib, matplotlib, numpy, os, pandas, pathlib, plotly, re, scipy, shutil, tempfile
Python features	NaN handling, SciPy, boolean masks / np.where, closures / nested functions, complex numbers, comprehensions, f-strings, generator expressions, importlib, lambda, linear algebra (np.linalg), matplotlib, pandas, pathlib, plotly, regular expressions, try/except

Class / function	What it does (first docstring line)
`find_silver(cond)`	The silver directory for this condition, or None. See result_dir().
`decompose(f, Zg, Zl)`	Zl = a*Zg + R + jwL with a, R, L real — separates the three causes.
`_voigt_basis(freq, taus)`	Complex design matrix for Voigt ladder with fixed time constants.
`lin_kk(freq, Z, M, add_inductance, c_crit)`	Linear KK test. Returns dict with residuals and fitted model.
`_voigt_basis(freq, taus)`	—
`drt_tikhonov(freq, Z, lam, n_tau_per_decade, extend_decad…)`	Deconvolve gamma(ln tau) from impedance spectrum.
`drt_peaks(res, min_rel_height)`	Find peak timescales and their areas (= resistance of each process).
`gamry_series_L(f, Z, top_decade)`	—
`hfr_band(f, Z, frac)`	HF side of the arc: from where -Z'' falls to `frac` of its peak, to f_max. This is the region that determines the ohmic intercept.
`r_ohmic_topband(f, Z, decades)`	Mean Re Z over the top third of a decade — the estimator silver uses, applied identically to both sides.
`interp_log(f_new, f, Z)`	—
`_row_A(r)`	Method A result (params in Ohm*cm2) -> fast/slow in mOhm*cm2.
`_row_B(e)`	—

eis_paths.py

40 lines · 0 classes · 1 functions

Purpose. Inserts the package root and the six sub-folders (core, pipeline, checks, analysis, plotting, readers) at the front of sys.path, once.

Why it exists. Lets every module import every other one by plain name (import silver, from config import DEFAULT) no matter which folder it lives in. This keeps the code compatible with the earlier flat layout and with Databricks notebooks.

Inputs → outputs. Nothing → modified sys.path (side effect at import time).

Depends on	only standard / third-party libraries
Used by	Local EIS Pipeline Runner, main, run
Libraries	pathlib, sys
Python features	comprehensions, pathlib, sys.path, type hints

Class / function	What it does (first docstring line)
`add(root)`	Insert root and its sub-folders at the front of sys.path (once).

run.py

37 lines · 0 classes · 1 functions

Purpose. Command-line launcher: python run.py <module> [args] imports the named module with importlib and calls its main(argv).

Why it exists. One uniform entry point for every module that has a main(), without having to cd into sub-folders.

Inputs → outputs. Command-line arguments → exit code of the module's main().

Depends on	eis_paths
Used by	—
Libraries	importlib, sys
Python features	f-strings, if __name__ == "__main__", importlib, raising exceptions, type hints

Class / function	What it does (first docstring line)
`main(argv)`	—

6.2 core/

Shared foundations used by every other folder: the configuration, numerical utilities, the plate geometry, plate statistics and plate conditions.

core/config.py

1,539 lines · 1 classes · 1 functions

Purpose. Every tunable number of the pipeline in one frozen dataclass Config, with the default instance DEFAULT. Also holds physical plate constants, FAMOS file-name patterns, presentation constants (colours, heat-map limits, port layout) and the fault rules. Provides replace(), preset(), to_dict(), save(), and constructors from JSON, environment variables and the command line.

Why it exists. Earlier versions had diverging copies of the same constants in four files, so results could not be reproduced. Now there is exactly one definition of each number and nothing downstream defines its own default. Frozen, so a config cannot change in the middle of a run.

Inputs → outputs. Defaults, JSON, EIS_* environment variables, CLI flags → an immutable Config object passed to every stage.

Depends on	only standard / third-party libraries
Used by	Local EIS Extra Analyses, Local EIS Pipeline Runner, bronze, csv_pipeline, gold, main, plate_figure, plate_plotly, plate_style, polcurve, silver, validate_hf (+ 27 test files)
Libraries	argparse, dataclasses, json, os, pathlib
Python features	@classmethod, JSON, argparse CLI, classes, comprehensions, dataclasses, f-strings, frozen dataclass, generator expressions, if __name__ == "__main__", lambda, pathlib, raising exceptions, type hints

Class / function	What it does (first docstring line)
`class Config @dataclass(frozen=True)`	One run of the pipeline. Methods: replace, famos_pattern, famos_patterns, preset, f_hi, f_eval_max, f_known_hi, to_dict, save, from_json, from_env, _coerce, build_argparser, from_cli.
`heatmap_limits(param)`	Fixed (vmin, vmax) in display units for `param`, or None = automatic.

core/plate_conditions.py

486 lines · 2 classes · 9 functions

Purpose. Turns port measurements into per-segment fields: temperature (measured by 4 sensors), pressure (interpolated inlet→outlet) and relative humidity (modelled water balance along the cathode channel). Every field carries a provenance label.

Why it exists. Lets impedance maps be compared with operating conditions on the same geometry, while stating honestly what is measured and what is modelled.

Inputs → outputs. Bench port state + plate temperatures → Field objects per parameter.

Depends on	utils
Used by	main
Libraries	dataclasses, numpy
Python features	@property, NaN handling, classes, closures / nested functions, comprehensions, dataclasses, f-strings, generator expressions, lambda, type hints

Class / function	What it does (first docstring line)
`class PortState @dataclass`	The operating point at the ports, in SI-ish units the bench reports.
`class Field @dataclass`	One scalar field over the segments, and how much of it was measured. Methods: measured, summary.
`saturation_pressure_pa(t_c)`	Saturation vapour pressure of water, Buck (1996).
`dew_point_to_mole_fraction(dew_c, p_total_pa)`	Vapour mole fraction from a dew point and a total pressure.

Class / function	What it does (first docstring line)
`cathode_stoichiometry(current_a, dry_air_nl_min, o2_fraction)`	Air supplied / air needed, lambda.
`port_state_from_bench(state, plate_t)`	Turn a `gamry_compare.BenchLog.state_at()` reading into a PortState.
`_flow_coordinate(centroids, axis, inlet)`	Position of each segment along the flow path, 0 at the inlet, 1 at the outlet.
`_current_fraction(xi, areas, j_dc)`	Fraction of the total current produced upstream of each segment.
`condition_fields(centroids, areas, ports, temp_sensor_x_mm, j…)`	Per-segment temperature, pressure and relative humidity.
`_humidity_field(xi, areas, ports, t_field, p_field, j_dc)`	—
`_selftest(log)`	—

core/plate_stats.py

173 lines · 0 classes · 9 functions

Purpose. Plate mean, median and standard deviation over 36 equal tiles of 5×5 pads, with segment k = 1…36 standing for tile k.

Why it exists. Equal-area statistics: the denser segmentation at the plate ends would otherwise pull the plate mean towards the inlet and outlet.

Inputs → outputs. {segment: value} → tile statistics, summary line, plate_statistics.csv.

Depends on	r2d2_geometry
Used by	Local EIS Pipeline Runner, csv_pipeline, gold, plate_figure, plate_maps, plate_plotly, plausibility (+ 1 test files)
Libraries	numpy, pandas, pathlib
Python features	NaN handling, comprehensions, f-strings, generator expressions, pandas, pathlib, raising exceptions, try/except, type hints

Class / function	What it does (first docstring line)
`tile_of(seg)`	(col0, col1, row0, row1), 1-based inclusive pad indices of the tile segment seg (1..36) represents.
`tile_bounds_mm(seg)`	(x0, x1, y0, y1) of the tile in mm, origin at the top-left pad.
`covered(seg, plate_name)`	The other segments with pads inside this tile -- the ones whose values the tile statistics ignore.
`_finite(values)`	—
`tile_values(values, classes, positive)`	segment -> value for the representative segments that count: finite, measured (class '' or 'measured' when classes are given) and, with `positive`, > 0 (a non-positive resistance is a failed fit).
`tile_stats(values, classes, positive)`	n, mean, median, sd, CV [%], min, max, span [%] over the 36 tiles.
`summary_line(values, classes, dec, unit, sep)`	'35 of 36 tiles · mean .. · median .. · sd .. · CV .. · min..max ..'
`statistics_rows(summary)`	One row per parameter of a plate_summary table (a DataFrame or the path of plate_summary.csv, display units): n, mean, median, sd, CV, min, max over the 36 tiles.
`write_statistics(summary_csv, out_csv)`	plate_summary.csv -> plate_statistics.csv beside it (or `out_csv`). Returns the path, or None when there was nothing to summarise.

core/r2d2_geometry.py

912 lines · 2 classes · 21 functions

Purpose. Geometry of both R2-D2 plate generations: which of the 900 pads belong to which of the 72 segments, segment areas and centroids, label positions, temperature interpolation along the flow and a self-check. use_plate() switches the active plate for the whole process.

Why it exists. Segments are not equal in size (1.36–8.47 cm² on gen1), and gen1/gen2 group the edge pads differently. Every area-weighted sum depends on getting this right, so it lives in one verified place.

Inputs → outputs. Plate name → Segment objects, areas, centroids, plate map drawings, segment_areas_<plate>.csv.

Depends on	plate_style
Used by	Local EIS Pipeline Runner, bronze, card_gain, csv_pipeline, csv_source, dc_closure, eis_local, gold, main, neighbours, nyquist, plate_figure, plate_maps, plate_plotly, plate_stats, plausibility, silver, utils, viewers (+ 14 test files)
Libraries	dataclasses, matplotlib, pathlib
Python features	@classmethod, @property, classes, closures / nested functions, comprehensions, context managers (with), dataclasses, f-strings, frozen dataclass, generator expressions, global state, if __name__ == "__main__", lambda, matplotlib, pathlib, raising exceptions, try/except, type hints

Class / function	What it does (first docstring line)
`class Segment @dataclass(frozen=True)`	One measured segment: the set of pads wired to it. Methods: from_rect, from_pads, name, n_pads, area_cm2, col0, col1, row0, row1, is_rectangle, x0_mm, x1_mm, y0_mm, y1_mm, w_mm, h_mm, cx_mm, cy_mm, runs.
`class Plate @dataclass(frozen=True)`	Methods: title.
`_parse_matrix(text)`	—
`_segments_from_matrix(grid)`	—
`_label_pads(segs)`	A pad inside the segment to hang its number on.
`_build_gen1()`	—
`_build_gen2()`	—
`plate_key(name)`	Normalise any spelling of a plate to its registry key.
`plate(name)`	—
`use_plate(name)`	Select the plate generation for the whole process.
`active_plate()`	—
`areas(plate_name)`	Segment name -> active area in cm^2 (active plate unless given).
`equal_areas(plate_name)`	Every segment given the same area, A_CELL/72 = 4.235 cm^2.
`label_xy(plate_name)`	Segment name -> (x, y) in mm of the centre of its label pad -- the red number box on the drawing, where the segment is drawn on the maps.
`centroids(plate_name)`	Segment name -> (x, y) centroid in mm, origin at the top-left pad.
`areas_from_csv(path)`	Read an override table. Accepts 'segment,area_cm2' with , or ; and an optional header. Missing segments fall back to the built-in value of the ACTIVE plate.
`temperature_at(x_mm, sensor_T)`	Linear interpolation of the four plate temperature sensors in x.
`segment_temperatures(sensor_T, plate_name)`	—
`renumbering(src, dst)`	Best-effort map src segment number -> dst segment number, by centroid.
`symmetry_report(plate_name)`	Does the pad map mirror onto itself, left-right and top-bottom?
`self_check(verbose, plate_name)`	Assert that the reconstruction tiles the plate exactly.
`write_csv(path, plate_name)`	—
`plot_map(path, value, label, title, plate_name, limits)`	Draw the plate. Colour = `value` per segment, default = area.

core/utils.py

1,119 lines · 2 classes · 35 functions

Purpose. Shared toolbox: IEEE-1057 sine fits (fit3, fit4), the joint two-channel fit (fit7_joint), harmonic distortion, stationarity, Cramér–Rao uncertainty (crlb_phasor, sigma_rel_from_snr), |Z| outlier detection, delay helpers (apply_delay), geometric-grid fit, complex interpolation, gain-file loading, console-safe logging, CSV/JSON writers, and segment_areas(cfg).

Why it exists. Anything used by more than one layer is defined once here, so no layer carries its own copy and every estimator can be unit-tested in isolation.

Inputs → outputs. NumPy arrays / dicts → phasors, uncertainties, masks, files on disk, log lines.

Depends on	r2d2_geometry
Used by	Local EIS Pipeline Runner, bronze, card_gain, csv_pipeline, csv_source, dc_closure, frequency_response, gamry_compare, gold, hf_response, main, neighbours, plate_conditions, plausibility, segment_scale, silver, validate_hf (+ 6 test files)
Libraries	contextlib, csv, dataclasses, io, json, logging, numpy, pathlib, re, sys, time
Python features	@property, JSON, NaN handling, boolean masks / np.where, classes, closures / nested functions, complex numbers, comprehensions, context managers (with), dataclasses, f-strings, generators (yield), global state, if __name__ == "__main__", inheritance, lambda, linear algebra (np.linalg), logging, np.errstate, pathlib, raising exceptions, regular expressions, try/except, type hints

Class / function	What it does (first docstring line)
`class _SafeStreamHandler`	A handler no console character can break. Methods: emit.
`class JointFit @dataclass`	Result of the seven-parameter two-channel fit. Methods: ratio, phase_diff_rad.
`_console_stream()`	stdout, put into UTF-8 if the platform allows it.
`_rules_for(stream)`	The heavy and light rule characters this stream can actually carry.
`get_logger(verbose)`	—
`banner(text, log)`	—
`section(text, log)`	—
`timed(text, log)`	—
`_design3(t, f, ramp)`	[cos, sin, 1] and optionally a centred ramp column.
`fit3(y, fs, f, detrend)`	Three-parameter fit y ~ a cos(wt) + b sin(wt) + c at KNOWN f.
`fit4(y, fs, f0, n_iter, detrend)`	Four-parameter fit: frequency solved for as well (IEEE Std 1057).
`fit7_joint(y_ref, y_sig, fs, f0, n_iter, tol, detrend)`	Seven-parameter sine fit of two channels sharing ONE frequency.
`background_at(xq, fs, f, n, n_win, off_bins)`	What a channel shows at frequency `f` while nothing is excited.
`harmonic_distortion(y, fs, f)`	THD from the 2nd and 3rd harmonic -- the linearity check.
`stationarity(y, fs, f, n_sub)`	Spread of the phasor over `n_sub` sub-windows -- the time-invariance check. Each sub-fit uses its own t=0, so the phasors are rotated back to a common origin before comparison; otherwise a perfectly stationary sine looks
`crlb_phasor(snr_db, n, freq_known)`	Cramer-Rao lower bounds for a single tone in white Gaussian noise.
`sigma_rel_from_snr(snr_db, n, model, floor, ceiling)`	Per-point relative standard deviation of |Z|, for KK weighting.
`zmag_outliers(freq, Z, n_mad, win)`	Flag points whose |Z| departs from the segment's own local trend.
`combine_snr_db(snr_a, snr_b)`	SNR of a ratio of two independent phasors, in dB.
`unwrap_phase_monotone(freq, Z)`	Phase of Z, unwrapped in order of increasing frequency.
`apply_delay(freq, Z, dt)`	Undo an acquisition skew of `dt` seconds.
`all_pass_residual(freq, Z)`	How much all-pass content the spectrum still carries.
`signed_delay_from_phase(freq, Z, decades)`	Delay from the high-frequency phase slope, with its sign and quality.
`hf_phase_slope(freq, Z, decades)`	d(phase)/d(ln f) over the top `decades` of the band, in rad.

Class / function	What it does (first docstring line)
`moving_mean(x, win)`	—
`runs_of_true(mask)`	Contiguous True runs of a boolean array as (start, stop) pairs.
`geometric_grid_fit(freqs, tol)`	Recover a sweep's geometric grid f_k = f0 * r^-k from its frequencies alone.
`interp_complex(f_new, f, Z)`	Log-frequency interpolation of a complex spectrum.
`extrap_complex(f_new, f, Z, extrapolate)`	—
`nan_safe_stats(x)`	—
`read_pairs(path)`	Read a two-column calibration file ('c0;c1' per line).
`load_gain(path)`	Ex-situ frequency response of the measuring chain.
`gain_at(gain, seg, freq)`	Interpolate the chain response for one segment onto `freq`.
`write_json(path, obj)`	—
`write_table(path, rows, columns)`	Minimal CSV writer -- no pandas dependency in the core path.
`_selftest(seed)`	Synthetic checks of the estimators. Run: python utils.py
`segment_areas(cfg)`	The area table this run should use.

6.3 pipeline/

The processing itself: the orchestrator (main.py), the three layers bronze, silver, gold, the separate CSV pipeline, and the signal-processing modules they call.

pipeline/main.py

655 lines · 0 classes · 9 functions

Purpose. The orchestrator and command-line interface. run_pipeline(cfg) selects the plate, then runs bronze → silver → gold → checks and writes run_manifest.json. reevaluate() reruns silver and gold from saved bronze CSVs. self_test() runs every check that needs no measurement data.

Why it exists. One function that the notebook, the CLI and the tests all call, so a run is the same computation wherever it starts.

Inputs → outputs. Config → run folder (bronze/, silver/, gold/, checks) + manifest dict.

Depends on	bronze, card_gain, config, csv_pipeline, csv_source, dc_closure, eis_paths, frequency_response, gamry_compare, gold, plate_conditions, plausibility, r2d2_geometry, silver, utils, validate_hf
Used by	Local EIS Pipeline Runner (+ 2 test files)
Libraries	argparse, json, pathlib, sys, time
Python features	JSON, argparse CLI, comprehensions, f-strings, generator expressions, if __name__ == "__main__", pathlib, raising exceptions, sys.path, try/except, type hints

Class / function	What it does (first docstring line)
`build_parser()`	—
`config_from_args(a)`	—
`geom_area(cfg)`	The cell area the comparison must use.
`run_pipeline(cfg, stop_after)`	bronze -> silver -> gold, with a manifest tying the three together.
`reevaluate(run_dir, out_dir)`	Silver -> gold -> plausibility again, on a finished FAMOS run.
`_reapply_card_gain(br, cfg, log)`	Bring the saved Z to what cfg.card_gain asks for.
`whole_cell_and_plausibility(cfg, manifest, log, sr)`	Gamry cross-check, then plausibility -- shared by FAMOS and CSV paths.
`self_test()`	Everything that can be checked without measurement data.
`main(argv)`	—

pipeline/bronze.py

2,484 lines · 4 classes · 35 functions

Purpose. Layer 1. Finds the card files, inventories every channel with its ADC slot, loads the calibration, aligns the separately-armed cards by cross-correlation, builds one consensus excitation schedule for the plate, refines the frequency per card, measures the card voltage gain and computes the raw impedance Z = K·A_U/A_seg per segment and step. Saves and reloads its tables.

Why it exists. Reading the raw waveforms is the slow part. Bronze does it once and stores only measured values, so every later correction can be rerun from bronze in seconds and switched off for comparison.

Inputs → outputs. FAMOS .DAT files (or datago shim) + curr.csv/temp.csv + gain file → BronzeRun object and bronze/*.csv.

Depends on	card_gain, config, eis_local, gamry_sync, hf_schedule, ladder_snap, r2d2_geometry, tone_estimation, utils
Used by	Local EIS Pipeline Runner, main, silver (+ 18 test files)
Libraries	csv, dataclasses, fractions, hashlib, json, numpy, pathlib, re, scipy, sys
Python features	@property, FFT (np.fft), JSON, NaN handling, SciPy, boolean masks / np.where, classes, closures / nested functions, complex numbers, comprehensions, context managers (with), csv module, dataclasses, f-strings, generator expressions, hashing, if __name__ == "__main__", lambda, linear algebra (np.linalg), np.errstate, pathlib, raising exceptions, regular expressions, try/except, type hints

Class / function	What it does (first docstring line)
`class ChannelInfo @dataclass`	One physical channel on one card. Methods: slot_seconds.
`class BronzeSpectrum @dataclass`	Raw spectrum of one segment: what the instrument actually reported. Methods: n_points, slot_delta_seconds, j_dc.
`class CardInfo @dataclass`	—
`class BronzeRun @dataclass`	Everything bronze knows. Serialisable, cacheable, re-runnable. Methods: segments_measured, segments_missing, summary, coverage_summary.
`discover_files(cfg)`	FAMOS files for THIS RUN, sorted so card order is deterministic.
`_digest(items)`	—
`inventory_channels(files, cfg, log)`	Open every card once and record what is on it, including slot order.
`cfg_stride(cfg)`	Decimation used for cheap statistics (std, DC level).
`estimate_card_lags(files, cards, cfg, log)`	Sample offset of every card relative to the first, from the reference.
`card_ratio(info)`	—
`card_shift(info)`	—
`to_own(idx, info)`	Common-base sample index -> this card's own sample index.
`to_common(idx, info)`	This card's own sample index -> common-base sample index.
`fs_common(lags, fs_seen)`	Sample rate of the common base: the anchor's.
`_to_rate(x, fs_from, fs_to)`	Polyphase resampling of an alignment trace onto another rate.
`_guard_samples(cfg, fs)`	The prominence guard for this card, in samples.
`_alignment_trace(stem, cards, cfg)`	The card's reference, band-passed to the band the lag is measured in.
`_best_lag(x, ref, max_lag, guard)`	The lag of best agreement, its correlation, and its prominence.
`_prominence(mag, k, guard)`	How many robust sigma the peak stands above the rest of the curve.
`_agreeing_cards(stem, lag, candidates, cards, cfg)`	Which other cards independently returned the same offset.
`_blockwise_lag_diagnostic(x, ref, global_lag, fs, cfg)`	Does ONE constant lag hold for the whole record, or do the clocks drift?
`_pairwise_closure(stems, traces, lags, cards, cfg, guard)`	Do the anchor-relative lags agree with direct card-to-card lags?

Class / function	What it does (first docstring line)
`_pick_anchor(stems, traces, max_lag, log, guard)`	The card whose clock the others are shifted onto: the one they agree with best.
`consensus_schedule(files, cards, cfg, log, lags)`	One excitation schedule for the whole plate.
`bronze_frequency_coverage(run_obj)`	How much of the plate actually carries each frequency.
`coverage_summary(rows, min_area_coverage)`	What the coverage table says in one line, for the manifest.
`_finite_median(values)`	Median of the finite values, or NaN when there are none.
`_gamry_guided(kept, tl, files, cards, cfg, lags, fs_seen, …)`	gamry_sync on the consensus schedule: report, exact frequencies, and (guide) misplaced / missing windows re-located in their Gamry slot.
`_on_grid(f, grid, tol)`	—
`refine_frequencies(files, cards, schedule, grid, cfg, lags, log)`	One exact frequency per step, used by EVERY fit that follows.
`pooled_reference_phasors(files, cards, schedule, cfg, lags, log)`	One cell-voltage phasor per step, averaged over every card.
`_sensor_key(channel_name)`	FAMOS channel name -> calibration key.
`plate_temperatures(files, cards, cal, cfg, log)`	One temperature field for the whole plate, from every sensor on every card.
`_K_for(seg, cal, T, median_c0, median_c1, cfg)`	Transfer coefficient, imputing from the plate median if necessary.
`_fix_polarity(Z, freqs, snr_db, cfg)`	Return Z or -Z, deciding on the low-frequency, high-SNR points.
`process_card(fp, cal, schedule, grid, cfg, log, T_seg, la…)`	Raw phasors for every segment on one card.
`run(cfg, log)`	Ingest every card into one BronzeRun.
`load(run_dir)`	Rebuild a BronzeRun from the tables save wrote.
`save(run_obj, cfg, log)`	Write bronze tables so silver can be re-run without touching .DAT.

pipeline/silver.py

2,687 lines · 4 classes · 36 functions

Purpose. Layer 2. Leaves out segments whose DC current disagrees with their neighbours, fits the structural ADC skew per card, estimates each segment's chain lag, corrects the card phase, applies 11 point gates with a rejection ledger, models each spectrum (Bayesian DRT and the standard circuit), reads R_ohmic, validates with Lin-KK and Z-HIT, assigns tiers A/B/C, and builds the DC closure and the cell aggregate.

Why it exists. Every correction and every judgement lives here, separate from the raw data, so each can be inspected, switched off and compared. Nothing is deleted silently: the ledger records which gate removed each point.

Inputs → outputs. BronzeRun → SilverRun and silver/*.csv (spectra_clean, segments_summary, point_rejections, …).

Depends on	bronze, card_gain, channel_lag, config, eis_measurement_model, eis_validation, gamry_compare, neighbours, r2d2_geometry, standard_eis, utils
Used by	Local EIS Pipeline Runner, csv_pipeline, gold, main, plausibility, validate_hf (+ 12 test files)
Libraries	dataclasses, numpy, pathlib, scipy
Python features	@property, NaN handling, SciPy, boolean masks / np.where, classes, closures / nested functions, complex numbers, comprehensions, context managers (with), dataclasses, f-strings, generator expressions, lambda, linear algebra (np.linalg), np.errstate, pathlib, try/except, type hints

Class / function	What it does (first docstring line)
`class SkewModel @dataclass`	Structural acquisition-skew model for one card. Methods: slot_ratio, dt_for.
`class SilverSpectrum @dataclass`	One segment, corrected, modelled and graded. Methods: j_dc, R_ohmic_cv.
`class SilverRun @dataclass`	Methods: card_factors, reach, tiers.

Class / function	What it does (first docstring line)
`class FilledSpectrum`	A segment reconstructed from its neighbours.
`_kk_residual(freq, Z, sigma_rel, M)`	Weighted KK-model residual: how far the spectrum is from realisable.
`basis_value(basis, slot_seg, slot_ref)`	Channel-order term of the skew model, under either hardware hypothesis.
`skew_cost(prep, dt0, k, M, basis)`	Mean normalised KK residual of a card under dt = dt0 + k*basis.
`fs_prep(prep, default)`	Sampling rate implied by the prepared spectra (for delay bounds).
`optimise_skew(prep, k_nom, cfg, basis)`	Coarse-to-fine 2-D search for (dt0, k).
`fit_structural_skew(card, items, cfg, log, basis_override)`	Fit dt(slot) = dt0 + k * (slot_seg - slot_ref) for one card.
`_dt0_identifiability(prep, k, k_nom, cfg, basis)`	Is the common-mode delay visible above the noise in the cost?
`residual_skew_check(spectra, cfg, log)`	After correction, is there any channel-order structure LEFT?
`sp_slot(sp)`	—
`choose_global_basis(by_card, cfg, log)`	Decide the converter architecture ONCE for the whole plate.
`_prepare_card(items, cfg)`	Shared preparation used by both the basis vote and the per-card fit.
`fit_per_card_skew(card, items, cfg, log)`	Legacy single-delay-per-card model, kept for A/B comparison.
`_drt_design(freq, tau, with_L)`	[1, (jw), 1/(1+jw tau_m) ...] -- linear in every parameter.
`_se_kernel(x, sigma_f, ell)`	—
`bayesian_drt(freq, Z, sigma_rel, cfg)`	Gaussian-process DRT with a closed-form posterior.
`cfg_len(cfg)`	—
`r_ohmic_topband(freq, Z, sigma_rel, span_decades, n_min, n_max)`	R_inf as an inverse-variance weighted mean of Re Z at the top of band.
`r_ohmic_method(cfg)`	'intercept' or 'topband' -- config.r_ohmic_method, 'auto' resolved by config.full_band.
`r_ohmic_intercept(freq, Z, sigma_rel, f_min_hz)`	The high-frequency intercept: Re Z where Im Z changes from negative (capacitive, below) to positive (inductive, above), linear between the two points around it. The LOWEST such crossing above `f_min_hz` is the one that c
`r_ohmic_axis_fit(freq, Z, n_top)`	R_inf by extrapolating the top of the arc to Im Z = 0.
`extrapolate_hf(drt, f_hi, n)`	Evaluate the posterior above the measured band.
`_hf_response(path)`	utils.load_gain of the frozen high-frequency response, read once.
`gate_points(sp, cfg)`	The point-level gates of process_segment, on their own.
`process_segment(sp, skew, cfg, log, ledger, chain, card_phas…)`	De-skew, weight, model, validate and grade one segment.
`channel_lag_items(spectra, skew, cfg)`	segment -> (freq, de-skewed Z, usable) for channel_lag.estimate.
`card_hf_phase(spectra, skew, lags, cfg, log)`	Per-card phase offset above the phase-coherent limit (full band only).
`_segment_phase_items(spectra, skew, lags, card_ph, cfg)`	(freq, corrected Z, keep, fs) per segment: the de-skew, the channel lag and the card phase, exactly as process_segment applies them.
`gamry_card_phase(spectra, skew, lags, cfg, log)`	Each card's phase calibrated against the condition's Gamry sweep.
`dc_closure(spectra, cfg)`	Sum(j_s * A_s) against the load current.
`cell_aggregate(spectra)`	Area-weighted harmonic mean -- what a cell-level instrument would see.
`build_filled_spectra(spectra, cfg, log)`	Rebuild substituted and (optionally) missing segments from neighbours.
`cell_aggregate_full(spectra, filled, a_cell)`	The cell curve, with the arithmetic written out and both normalisations.
`dc_share_faults(spectra, cfg)`	Segments whose DC current density says the CHANNEL is broken.
`run(bronze_run, cfg, log)`	—

Class / function	What it does (first docstring line)
`uc_gain_pct(card_ref)`	card -> gain in % of the UC channel its Z divides by, AFTER whatever correction bronze applied (so 0 after a correction).
`save(sr, cfg, log)`	—

pipeline/gold.py

786 lines · 2 classes · 11 functions

Purpose. Layer 3. Collects per-segment parameters (R_ohmic, R_ct, R_mt, R_pol, Re Z at 1 kHz, |Z| and phase at 100 Hz, j, T), optionally fills unmeasured segments from a Gaussian-process field over the plate, labels faults by ratio to the plate median, computes 36-tile statistics and draws the plate maps and Nyquist plots.

Why it exists. Answers the question the rig exists for: where on the plate is something wrong, and what is it.

Inputs → outputs. SilverRun → GoldRun and gold/*.csv, plate_*_interp.png, nyquist.png/html.

Depends on	config, nyquist, plate_figure, plate_plotly, plate_stats, r2d2_geometry, silver, utils
Used by	Local EIS Pipeline Runner, csv_pipeline, main (+ 6 test files)
Libraries	dataclasses, matplotlib, numpy, pathlib
Python features	NaN handling, boolean masks / np.where, classes, closures / nested functions, comprehensions, dataclasses, f-strings, generator expressions, lambda, linear algebra (np.linalg), matplotlib, pathlib, try/except, type hints

Class / function	What it does (first docstring line)
`class SegmentRecord @dataclass`	One segment on the finished plate, measured or inferred.
`class GoldRun @dataclass`	Methods: measured, inferred.
`split_processes(sp, cfg)`	Bucket the DRT by time constant into ohmic / charge-transfer / transport.
`spatial_trends(records, params, plate_w_mm)`	Is a pattern on the map a gradient, or is it the segment geometry?
`collect_parameters(sr, cfg)`	parameter -> {segment: value}, and the matching uncertainties.
`re_at(freq, Z, f0)`	Re Z at f0 from the kept points, interpolated in log f; nan when f0 is outside them or the nearest points are more than a third of a decade away (no extrapolation).
`_kernel(X1, X2, lx, ly, kind)`	Anisotropic kernel on plate coordinates, in mm.
`fit_field(values, sds, cfg)`	Gaussian-process regression over the segment centroids.
`label_faults(records)`	Ratio-to-median rules, applied in place.
`plate_heatmap(records, param, cfg, title_extra)`	Static plate map with every segment drawn, measured or inferred.
`plate_heatmap_interactive(records, param, cfg)`	Plotly version: same map, with per-segment hover carrying the caveats.
`run(sr, cfg, log)`	—
`save(gr, sr, cfg, log)`	—

pipeline/csv_pipeline.py

1,712 lines · 1 classes · 25 functions

Purpose. A complete second pipeline for the single-clock CSV logger: tone detection, R2-D2 logger point files with channel-scan de-skew and alias detection, phasors on recorded timestamps, impedance, Lin-KK, ZARC circuit fit with AICc arc count, aggregate and outputs.

Why it exists. A CSV logger has one clock, so the FAMOS synchronisation stage would estimate a quantity that does not exist. It has different problems (timestamp jitter, text quantisation, channel scan) that need their own treatment.

Inputs → outputs. CSV file or sweep folder + calibration → same kind of products as bronze/silver/gold.

Depends on	channel_lag, config, csv_source, eis_local, eis_validation, gamry_compare, gold, nyquist, plate_figure, plate_stats, r2d2_geometry, silver, utils
Used by	Local EIS Pipeline Runner, main (+ 3 test files)
Libraries	argparse, dataclasses, json, matplotlib, numpy, pathlib, scipy, time
Python features	FFT (np.fft), JSON, NaN handling, SciPy, argparse CLI, boolean masks / np.where, classes, closures / nested functions, complex numbers, comprehensions, context managers (with), dataclasses, f-strings, generator expressions, if __name__ == "__main__", lambda, linear algebra (np.linalg), matplotlib, np.errstate, pathlib, raising exceptions, try/except, type hints

Class / function	What it does (first docstring line)
`class SegmentSpectrum @dataclass`	—
`_uniform_view(t, y, uniform, fs)`	y on a uniform grid -- FOR PEAK DETECTION ONLY.
`_peaks(y, fs, f_lo, f_hi, snr_db, dyn_range_db)`	Local maxima of the windowed amplitude spectrum, above TWO thresholds.
`detect_tones(m, cfg, log)`	Find the excitation frequencies in the cell-voltage record.
`_detect_sweep_windows(t, y, fs, uniform, f_lo, f_hi, cfg)`	Split a stepped sweep into (start, stop, freq) dwells.
`deskew(A, tau, freq, conjugated)`	Move one channel's phasor back to the row's nominal instant.
`nyquist_zone(phasors, taus, f_alias, fs, max_zone)`	Which analogue frequency does the channel scan say this point was at?
`r2d2_point(m, cfg, log, f_true)`	One point file -> one frequency and one phasor per channel.
`sweep_grid_check(freqs)`	Do the recovered frequencies form a real sweep?
`r2d2_sweep_spectra(points, cfg, log)`	Assemble the per-segment spectra from a folder of point files.
`apply_scan_skew(freq, Z, seg_slot, ref_slot, scan_rate_hz)`	Remove a KNOWN channel-scan offset.
`segment_spectra(m, sched, cal, T_seg, cfg, log)`	Z_s(f) = U_cell(f) / j_s(f) for every segment, with uncertainty.
`from_frequency_file(m, cfg, log)`	A CSV that already holds spectra: no phasor estimation at all.
`z_zarc(w, R, tau, n)`	One ZARC (R in parallel with a CPE), parameterised by (R, tau, n).
`z_model(p, freq, n_arcs)`	Rs + jwL + sum of ZARCs.
`ecm_start(freq, Z, n_arcs)`	Starting values read off the spectrum, not guessed.
`fit_ecm(freq, Z, sigma_rel, n_arcs, weight)`	Weighted complex non-linear least squares with parameter intervals.
`capacitive_band(freq, Z)`	Mask of the points between the HF and the LF zero crossing of Z''.
`choose_n_arcs(freq, Z, sigma_rel)`	Fit one arc and two, keep the one the data supports.
`cell_aggregate(spectra, areas, a_cell)`	Z_cell = A_cell / sum_s (A_s / Z_s) -- the segments are in parallel.
`validate(sp, cfg)`	Linear Kramers-Kronig residuals -- is the spectrum a spectrum?
`write_outputs(spectra, ecm, agg, cfg, log, extra, reader_m…)`	—
`run_csv(cfg, stop_after)`	The CSV path, end to end.

Class / function	What it does (first docstring line)
`_apply_channel_lag(spectra, cfg, log)`	channel_lag.estimate on CSV spectra; Z corrected in place. Returns the per-segment results and writes silver/channel_lag.csv.
`_seg_key(s)`	—
`_finish_csv(spectra, sched, m, cfg, log, t0, stop_after,…)`	Validation, ECM, aggregation and output — shared by every CSV route.

pipeline/channel_lag.py

332 lines · 1 classes · 11 functions

Purpose. Measures each segment's current-chain lag against the complex plate median (fit of the phase with a pure delay or first-order lag), accepts it only if the data follow the model, and returns the correction. Iterates three times.

Why it exists. Shunt/trace/amplifier chains lag by up to ~110 µs, which shifts R_ohmic by up to ~20 mΩ·cm². Measuring it on every run covers any plate and wiring.

Inputs → outputs. Per-segment spectra → {segment: ChannelLag} with tau, residual and status.

Depends on	only standard / third-party libraries
Used by	Local EIS Pipeline Runner, csv_pipeline, gamry_dta, silver (+ 1 test files)
Libraries	dataclasses, numpy, scipy
Python features	NaN handling, SciPy, classes, closures / nested functions, complex numbers, comprehensions, dataclasses, f-strings, generator expressions, lambda, try/except, type hints

Class / function	What it does (first docstring line)
`class ChannelLag @dataclass`	Methods: row.
`_opt(cfg, name, default)`	—
`mode(cfg)`	—
`_key(f)`	—
`lag_model(cfg)`	—
`correction(freq, tau_s, model)`	The factor that removes the channel's lag: Z_true = Z_meas * this.
`predicted_phase(freq, tau_s, model)`	—
`plate_reference(items)`	Complex plate median per frequency, from the usable points only.
`fit_tau(freq, phase, tau_max, model)`	(tau, rms residual in deg) for phase = predicted_phase(f, tau), |tau| <= tau_max. The residual is wrapped, so a delay that turns the top point past 180 deg is still fitted; a coarse grid finds the basin first.
`estimate(items, cfg, log)`	Lag per segment. `items`: segment -> (freq, Z, usable mask).
`_log_summary(out, m, f_lo, log)`	—
`card_summary(lags, card_of)`	Per card: how many segments, how many unreliable, median tau.

pipeline/eis_local.py

1,611 lines · 5 classes · 28 functions

Purpose. Library of the original stand-alone evaluator: FAMOS readers for both file dialects (FamosV1 float32, FamosV2 float64) behind the FamosFile factory, PlateCalibration (K(T), T(u)), gain-file loading, sine fits, blind step detection (detect_schedule) with dwell-window finding, DC closure and parallel aggregation.

Why it exists. These are the low-level building blocks bronze and the CSV path share. Keeping them in one module means one implementation of each reader and detector.

Inputs → outputs. File paths, waveforms → FamosFile objects, Step lists, K values, phasors.

Depends on	eis_validation, hf_schedule, r2d2_geometry
Used by	Local EIS Pipeline Runner, abgleich, bronze, card_gain, csv_pipeline, gamry_dta, hf_schedule, polcurve (+ 10 test files)

Libraries	argparse, dataclasses, json, matplotlib, numpy, pathlib, re
Python features	@classmethod, @property, FFT (np.fft), JSON, NaN handling, __new__ factory, __post_init__, argparse CLI, boolean masks / np.where, classes, complex numbers, comprehensions, context managers (with), dataclasses, f-strings, generator expressions, if __name__ == "__main__", lambda, linear algebra (np.linalg), matplotlib, np.errstate, np.memmap, pathlib, raising exceptions, regular expressions, try/except, type hints

Class / function	What it does (first docstring line)
`class FamosV1 @dataclass`	imc FAMOS **v1** binary reader -- one |CP carrying every channel name. Methods: __post_init__, channel, position, segment_names, uc_names, temp_names.
`class FamosV2`	imc FAMOS **v2**: one metadata block per channel, and float64 samples. Methods: channel, position, segment_names, uc_names, temp_names.
`class FamosFile`	Open a FAMOS recording, whichever dialect it is written in. Methods: __new__.
`class PlateCalibration @dataclass`	j_s [A/cm^2] = u_s [V] / (c0 + 0.001 * c1 * T[degC]) per segment T [degC] = (u_temp [V] - c0) / c1 per sensor Methods: load, has_current_cal, K, temperature.
`class Step @dataclass`	Methods: window_repaired, valid.
`_famos_plausible(fam)`	Empty string if this parse can be believed, else why it cannot.
`pick_reference_channel(fam, prefer, stride, log)`	The UC channel to use as this card's reference. Fixed by name.
`_read_pairs(path)`	—
`load_gain(path)`	Optional dynamic response of the measuring chain, from the ex-situ frequency-response characterisation.
`gain_at(gain, seg, freq)`	—
`fit3(y, fs, f)`	Three-parameter fit y ~ a*cos(wt) + b*sin(wt) + c at known f.
`fit4(y, fs, f0, n_iter)`	Four-parameter fit: also solves for the frequency (IEEE Std 1057).
`harmonic_distortion(y, fs, f)`	THD from the 2nd and 3rd harmonic; the linearity check.
`_moving_mean(x, win)`	—
`demod_envelope(x, fs, f, n_cyc, min_win_s, max_win_s)`	|complex amplitude at f| as a function of window start time.
`dwell_window(x, fs, f, centre, frac, trim)`	Contiguous interval around `centre` where the response at f is above `frac` of its peak: the interval in which that frequency was applied.
`polish_window(x, fs, f, a, b, iters)`	Trim the window until the fit residual stops improving.
`_runs(mask)`	Contiguous True runs of a boolean array as (start, stop) pairs.
`fill_gaps(ref, fs, steps, min_snr_db, max_rounds)`	Recover steps that the log grid stepped over.
`_collapse_overlapping(steps, min_overlap)`	Two steps cannot occupy the same stretch of the record.
`_dedupe(steps, rel_tol, overlap_tol)`	One physical step must yield one entry.
`_stationarity(x, fs, f, start, stop)`	—
`detect_schedule(ref, fs, ppd, f_lo, f_hi, min_snr_db, verbose)`	Find every stepped-sine dwell in the record without being told the frequency list.
`segment_spectrum(fam, uc_name, seg, steps, K, gain, uc_gain)`	Z_s(f) in ohm*cm^2, with per-step SNR.
`deskew(freq, Z, fam, uc_name, dt_range, verbose)`	Remove the acquisition skew between the reference and the segment channels, measured with Z-HIT.
`dc_closure(j_dc, areas, i_setpoint)`	sum_s j_s * A_s against the known cell current.
`aggregate(freq, Z, areas)`	Cell-level spectrum predicted from the local ones, in ohm*cm^2.
`_mpl()`	—
`plot_spectra(freq, Z, Z_cell, title, path)`	—
`plot_schedule(steps, fs, path)`	—

Class / function	What it does (first docstring line)
`plot_validation(val, path)`	—
`evaluate(dat_dir, out_dir, curr_cal, temp_cal, gain_f…)`	—
`main()`	—

pipeline/eis_measurement_model.py

493 lines · 0 classes · 9 functions

Purpose. The Boukamp/Agarwal measurement model: fits a Voigt (RC) ladder plus series R and L with fixed log-spaced time constants, returns the fit, its residual and parameters; also fits a delay together with the model and computes the parallel aggregate ASR.

Why it exists. A KK-compliant model is the best physically realisable description of the data; its residual is a quality measure and its R0 a high-frequency intercept constrained by every point.

Inputs → outputs. freq, Z, sigma → model curve, residual, R0, L, Rp; aggregate ASR.

Depends on	only standard / third-party libraries
Used by	Local EIS Pipeline Runner, silver (+ 2 test files)
Libraries	numpy
Python features	NaN handling, boolean masks / np.where, closures / nested functions, complex numbers, comprehensions, context managers (with), f-strings, if __name__ == "__main__", linear algebra (np.linalg), np.errstate, type hints

Class / function	What it does (first docstring line)
`has_inductive_tail(freq, Z, n_top)`	True when the spectrum is inductive at the TOP of its band: the mean Im Z of its n_top highest frequencies is > 0.
`_design(freq, taus, with_L)`	Complex design matrix [1, (jw), 1/(1+jw tau_k) ...].
`_weighted_lstsq(A, Z, w, ridge, n_free)`	Least squares on the complex system, stacked into real form.
`_mu(R)`	Schoenleber's mu: 1 - sum|R_neg| / sum R_pos. Falls towards 0 as the RC set degenerates into fitting noise.
`fit_kk_model(freq, Z, sigma_rel, M, mu_crit, with_L, tau_…)`	Fit R0 + jwL + sum R_k/(1+jw tau_k) with fixed log-spaced tau.
`fit_delay_and_model(freq, Z, sigma_rel, dt_range, n_grid, refine)`	Find the acquisition delay by minimising the KK-model residual.
`fit_shared_delay(spectra, dt_range, n_grid, M_scan, verbose)`	ONE delay for a whole card, fitted from all its segments at once.
`clean_segment(freq, Z, snr_db)`	Full cleaning chain for one segment. Z in ohm*cm^2.
`aggregate_asr(freq, Z_by_seg, areas, A_cell)`	Z_cell^ASR = A_cell / sum_s (A_s / Z_s^ASR).

pipeline/eis_validation.py

268 lines · 0 classes · 5 functions

Purpose. Instrument-independent validation: lin_kk (linear Kramers–Kronig test with Schönleber's μ criterion) and z_hit (reconstructing |Z| from the phase), plus a delay estimate.

Why it exists. Decides whether a spectrum is trustworthy without a second instrument. Z-HIT also sees pure time delays, which KK cannot.

Inputs → outputs. freq, Z → residual statistics and verdicts.

Depends on	only standard / third-party libraries
Used by	csv_pipeline, eis_local, silver (+ 1 test files)
Libraries	numpy
Python features	NaN handling, complex numbers, f-strings, if __name__ == "__main__", linear algebra (np.linalg), type hints

Class / function	What it does (first docstring line)
`_linkk_fit(freq, Z, M, add_inductance, add_capacitance)`	Least-squares fit of R0 [+ jwL] [+ 1/jwC] + sum_k R_k/(1+jw tau_k).
`_mu(R)`	Schönleber's mu: 1 - sum|R_neg| / sum|R_pos|.
`lin_kk(freq, Z, mu_crit, max_M, add_inductance, tol)`	Run the lin-KK test. Returns residuals and a pass/fail verdict.
`z_hit(freq, Z, gamma, fit_band)`	Reconstruct ln|Z| from the phase (Ehm/Göhr approximation).
`estimate_delay(freq, Z, dt_range, n_grid, fit_band)`	Find the pure time delay hidden in a spectrum, using Z-HIT.

pipeline/gamry_sync.py

612 lines · 2 classes · 14 functions

Purpose. Puts the FAMOS recording on the Gamry's clock using the .DTA timeline: measures the offset, gives steps the Gamry's exact frequencies, re-locates misplaced or missing windows (verified by the hf_schedule tests), confines low-frequency windows between step ends, and corroborates refused card lags. Modes: off / report / frequency / guide.

Why it exists. The FAMOS cards record the response to the Gamry's own sweep, so the Gamry log knows exactly when each frequency ran. Using it removes guesswork from window placement.

Inputs → outputs. Detected steps + Gamry .DTA → SyncResult, corrected steps, bronze/gamry_sync.csv.

Depends on	gamry_compare, hf_schedule
Used by	Local EIS Pipeline Runner, bronze, dc_closure (+ 2 test files)
Libraries	dataclasses, datetime, numpy, pathlib, re
Python features	classes, closures / nested functions, complex numbers, comprehensions, dataclasses, f-strings, generator expressions, lambda, pathlib, regular expressions, try/except, type hints

Class / function	What it does (first docstring line)
`class Timeline @dataclass`	One Gamry sweep in the order it was run.
`class SyncResult @dataclass`	Methods: summary.
`mode(cfg)`	—
`_num(s)`	—
`read_timeline(path)`	Freq and Time columns of a Gamry .dta ZCURVE table.
`read_vdc(path)`	Median of the Vdc column of a Gamry .dta ZCURVE table (the cell's DC voltage at the Gamry's own sense leads), nan when there is none.
`find_timeline(cfg, log)`	The sweep for cfg.condition under cfg.gamry_dir (same rules as the whole-cell comparison: order and build token decide the campaign).
`_match(f, freqs, tol)`	—
`align(steps, tl, fs, f_lo, f_hi, tol, min_snr_db, …)`	Offset between the FAMOS common time base and the Gamry clock, and a verdict for every Gamry point inside the analysis band.
`apply_frequencies(steps, sync, rebuild)`	Matched steps take the Gamry's exact frequency. Windows untouched.
`confine_windows(steps, sync, fs, rebuild, min_interval_s, gu…)`	Put every long (low-frequency) step's window inside its Gamry step.
`separate_windows(steps, fs, rebuild, min_cycles, log)`	No two windows may overlap: one stretch of record holds one tone.
`_lockin_power(chans, fs, f, a, b, hop, win)`	Array power at f in sliding windows over [a, b): sum over channels of |phasor|^2 -- sign-free, so a reversed sense pair cannot cancel.
`locate(chans, fs, f, lo, hi, win_cycles, min_dwell_…)`	The dwell of a tone at f inside [lo, hi): the contiguous run around the strongest array response where the power stays above a quarter of its peak, trimmed by 10 % at each end.
`relocate(steps, sync, tl, chans, fs, lag, rebuild, ma…)`	Re-locate misplaced and missing steps in their Gamry slot -- and every window that was only interpolated.

Class / function	What it does (first docstring line)
`corroborate_card_lags(per_card, lags, fs, tl, f_lo, f_hi, tol_s, f…)`	Check every card's own clock against the Gamry's.

pipeline/hf_schedule.py

1,145 lines · 5 classes · 14 functions

Purpose. Recovers the top decade of the sweep: stacks the polarity-aligned segment channels (their tone is flat in frequency), fits the ladder and verifies each predicted rung with two independent tests (CFAR channel count and a rank-1 statistic).

Why it exists. The cell voltage gets very small at high frequency, so a detector looking only at the voltage misses those steps; the segment channels still carry them.

Inputs → outputs. All channels of a card → HFSchedule with verified high-frequency steps.

Depends on	eis_local
Used by	bronze, eis_local, gamry_sync
Libraries	dataclasses, math, numpy
Python features	@property, FFT (np.fft), NaN handling, classes, closures / nested functions, complex numbers, comprehensions, context managers (with), dataclasses, f-strings, generator expressions, lambda, linear algebra (np.linalg), np.errstate, type hints

Class / function	What it does (first docstring line)
`class LazyChannels`	Read-through view of one card's segment channels. Methods: keys, window.
`class Ladder @dataclass`	f_k = f0 * ratio^-k, with dwell_s(f) = max(d_min, n_cyc / f). Methods: freq_of, index_of, tol_at, dwell_s.
`class CfarResult @dataclass`	Outcome of the frequency-domain CFAR at one candidate rung. Methods: p_false_alarm, accept.
`class Rank1Result @dataclass`	Outcome of the maximum-eigenvalue test at one candidate rung. Methods: usable, accept.
`class HFSchedule @dataclass`	What `recover_schedule` returns. Methods: as_steps, summary.
`crlb_usable(snr_db, n, sigma_rel_max, freq_known)`	Is a phasor from n samples at `snr_db` precise enough to keep?
`_names(chans)`	—
`_window(chans, name, a, b)`	One window out of one channel, without materialising the channel.
`_standardise(x)`	Zero mean, unit variance, NaNs removed.
`polarity_aligned_reference(chans, stride, log)`	Sum the segment channels into one virtual reference trace.
`_snap_ppd(ppd_free, rel_tol)`	Round the spacing to the nearest integer points/decade when it is close enough that a generator would have been asked for that number.
`fit_ladder(freqs, starts, stops, fs, tol, snap)`	Recover the sweep's geometric ladder and dwell law.
`prune_off_ladder(freqs, lad, tol)`	Boolean mask: which of `freqs` sit on the ladder.
`_binom_sf_threshold(m, p, alpha)`	Smallest c with P(Binomial(m, p) >= c) <= alpha.
`cfar_channel_count(chans, fs, f, a, b, n_bg, guard)`	Frequency-domain CFAR at `f`, over the channel array, one snapshot.
`rank1_statistic(chans, fs, f, a, b, n_snap)`	Maximum-eigenvalue detection across the channel array.
`_median_channel_snr_db(chans, fs, f, a, b, max_ch)`	SNR a SINGLE channel sees at this step, not the stack's.
`recover_schedule(chans, fs, f_lo, f_hi, ppd, min_snr_db, sigm…)`	Recover one card's excitation schedule from its segment channels.
`_verify_rung(chans, ref, fs, f_pred, lad, anchors, ladder…)`	Locate one predicted rung in time, then accept or reject it.

pipeline/ladder_snap.py

364 lines · 0 classes · 7 functions

Purpose. Fits the stepped sweep's geometric ladder (integer points per decade), snaps detected frequencies to exact rungs and repairs dwell windows that break the time order of a monotonic sweep.

Why it exists. A 1 % frequency error over a 0.25 s dwell costs about 16 dB of phasor SNR. The ladder is known structure, so enforcing it is safer than trusting each detection.

Inputs → outputs. Detected steps → snapped steps with repaired windows.

Depends on	only standard / third-party libraries
Used by	Local EIS Pipeline Runner, bronze (+ 2 test files)
Libraries	numpy
Python features	NaN handling, boolean masks / np.where, closures / nested functions, complex numbers, comprehensions, f-strings, generator expressions, lambda, try/except

Class / function	What it does (first docstring line)
`fit_ladder(freq, ppd_candidates)`	Points per decade from the MEDIAN log-spacing of the detections.
`ladder_phase(freq, n_samples, amp, ppd, fs, min_cycles, d…)`	Ladder offset in [0,1), from the cleanly resolved fixed-dwell steps.
`snap(freq, ppd, offset)`	Nearest rung frequency, distance to it in rung units, and rung index.
`snap_schedule(rows, ppd, offset, fs, amp_key)`	Collapse a detected schedule onto its ladder.
`snap_steps(steps, fs, ppd, offset, log)`	Snap a list of `eis_local.Step` onto the excitation ladder.
`_rebuild(cls, src)`	A copy of `src` with changes applied, keeping every other field.
`repair_windows(steps, fs, descending, min_dwell_frac, t_tol…)`	Replace dwell windows that cannot belong to the sweep.

pipeline/neighbours.py

460 lines · 2 classes · 9 functions

Purpose. Builds segment adjacency from shared pad edges, flags segments that differ from their neighbour ring (robust modified z-score), and rebuilds a segment's spectrum from its neighbours when asked.

Why it exists. A plate has a real inlet-to-outlet gradient; comparing with the immediate neighbours cancels the gradient but not a fault.

Inputs → outputs. Segment values or spectra → findings, donor rings, filled spectra.

Depends on	r2d2_geometry, utils
Used by	silver (+ 2 test files)
Libraries	core, dataclasses, numpy
Python features	@property, NaN handling, boolean masks / np.where, classes, complex numbers, comprehensions, context managers (with), dataclasses, f-strings, generator expressions, if __name__ == "__main__", lambda, np.errstate, raising exceptions, try/except, type hints

Class / function	What it does (first docstring line)
`class Finding @dataclass`	One segment, one parameter, one verdict. Methods: direction, as_row.
`class SpectrumFinding @dataclass`	Methods: worst, band_summary.
`adjacency(plate_name)`	Segment -> the set of segments sharing a pad edge with it.
`_modified_z(x, ring)`	(z, median, MAD-as-sigma) of x against `ring`.
`_as_int(name)`	—
`scalar_outliers(values, param, plate_name, z_cut, adj)`	Segments whose value stands out from the ring that touches them.

Class / function	What it does (first docstring line)
`spectrum_outliers(spectra, plate_name, adj, min_points)`	Per-frequency departure of each segment from its neighbour ring.
`donor_ring(seg, available, plate_name, max_hops)`	The nearest ring of AVAILABLE segments around seg.
`fill_spectrum(seg, spectra, areas, plate_name, max_hops)`	An area-specific spectrum for seg, built from its neighbours.
`analyse(params, spectra, plate_name, z_cut)`	Run every scalar parameter and, when given, the spectra.
`_self_test()`	Plant one fault and check it is the segment that comes back.

pipeline/standard_eis.py

283 lines · 0 classes · 9 functions

Purpose. The textbook PEMFC circuit fit R_ohmic + jωL + two ZARC arcs by complex non-linear least squares (scipy.optimize.least_squares) with bounds and many starting points; also the high-frequency intercept and the graphical polarisation resistance.

Why it exists. Gives R_ct/R_mt/R_pol the way ZView or Gamry Echem Analyst would, beside the DRT, so both evaluations come out of one run.

Inputs → outputs. freq, Z → dict of fitted parameters, residuals, R_ohmic and how it was read.

Depends on	only standard / third-party libraries
Used by	Local EIS Pipeline Runner, silver (+ 1 test files)
Libraries	numpy, scipy
Python features	NaN handling, SciPy, closures / nested functions, complex numbers, f-strings, generator expressions, linear algebra (np.linalg), try/except, type hints

Class / function	What it does (first docstring line)
`zarc(w, R, tau, n)`	R ‖ CPE, written with its time constant.
`z_model(p, freq)`	Z of R_ohmic + jωL + two ZARCs; p = (Rs, L, R1, tau1, n1, R2, tau2, n2).
`_to_p(x)`	Fit vector -> parameters. x = (Rs, L, R1, log10 τ1, n1, R2, Δlog10 τ, n2).
`graphical(freq, Z)`	Model-free numbers read straight off the spectrum.
`hf_intercept(freq, Z, n_top)`	R_ohmic from the high-frequency end, model-free.
`capacitive_band(freq, Z)`	Mask of the points the circuit describes: everything above the low-frequency inductive loop. At high current the spectrum turns inductive below ~1 Hz (Im Z > 0); the two-arc circuit has no element for that loop, so those
`_starts(f, Z)`	Starting vectors read off the spectrum.
`fit_spectrum(freq, Z, sigma_rel, rs_fixed)`	Fit the standard circuit to one spectrum.
`choose_n_arcs(freq, Z, sigma_rel)`	Drop-in for csv_pipeline.choose_n_arcs: always the standard two-arc circuit, so the notebook's ECM views show the same fit the maps use.

pipeline/tone_estimation.py

297 lines · 1 classes · 9 functions

Purpose. Finds one exact excitation frequency per card and step with a multichannel maximum-likelihood search (variable projection over all channels of the card), and fits phasors at that frequency (phasors_at).

Why it exists. The older per-pair Gauss–Newton frequency fit landed off the tone in about 30 % of trials. All channels of a card share one clock, so one frequency estimated from all of them is both correct and more precise.

Inputs → outputs. Window of all channels of a card → ToneEstimate (frequency), phasors per channel.

Depends on	only standard / third-party libraries
Used by	Local EIS Pipeline Runner, bronze (+ 2 test files)
Libraries	dataclasses, numpy, scipy
Python features	FFT (np.fft), NaN handling, SciPy, boolean masks / np.where, classes, closures / nested functions, complex numbers, comprehensions, context managers (with), dataclasses, f-strings, if __name__ == "__main__", linear algebra (np.linalg), np.errstate, raising exceptions, type hints

Class / function	What it does (first docstring line)
`class ToneEstimate @dataclass`	—
`_design(n, fs, f, ramp)`	—
`phasors_at(Y, fs, f, ramp)`	Three-parameter (+ramp) LS of every row of Y at ONE frequency.
`_rss(Y, fs, f, ramp)`	—
`_hann(n)`	—
`_periodogram_peak(Y, fs, f_lo, f_hi)`	Multichannel IpDFT: SNR-normalised summed Hann periodogram, 2-point Hann interpolation (Grandke 1983).
`card_frequency(Y, fs, f_prior, search_bins, ramp, prior_ban…)`	One frequency for all channels of one card in one dwell window.
`fit4(y, fs, f0, n_iter, detrend)`	IEEE-1057 four-parameter fit with the two defects removed.
`stationarity_test(y, fs, f, n_sub, alpha, ramp)`	Chi-square test of phasor constancy across n_sub sub-windows.
`_selftest(seed)`	—

6.4 checks/

Diagnostics that judge whether a finished result is believable. None of them can stop a run.

checks/card_diagnostics.py

295 lines · 0 classes · 10 functions

Purpose. Median Nyquist/Bode per card against the Gamry, and the card-to-Gamry transfer function across all currents.

Why it exists. Separates an acquisition problem (same at every current), a chain/comparison problem (shared by all cards) and a real cell effect.

Inputs → outputs. Run folders + Gamry → per-condition and cross-current figures.

Depends on	hf_response, nyquist
Used by	Local EIS Pipeline Runner (+ 1 test files)
Libraries	argparse, matplotlib, numpy, pandas, pathlib, re
Python features	NaN handling, argparse CLI, boolean masks / np.where, complex numbers, comprehensions, f-strings, if __name__ == "__main__", lambda, matplotlib, pandas, pathlib, raising exceptions, regular expressions, try/except, type hints

Class / function	What it does (first docstring line)
`_card(name)`	—
`_amps(cond)`	—
`card_medians(sp)`	card, fk, Z: complex median of the card's segments at each step; plus the plate median as card 'plate'.
`transfer(cm, gamry)`	H = Z_card / Z_Gamry at the Gamry's own point of each step.
`series_offset(cm, gamry, band, card)`	`card` (the plate median, or the cell aggregate -- the parallel sum of the segments, which is what the Gamry measures) minus the Gamry: median dRe and dIm over band, and dRe at the Gamry's own real-axis crossing (its H
`_axes_style(ax)`	—
`figure_condition(cond, cm, gamry, path)`	The four plots for one operating point.

Class / function	What it does (first docstring line)
`figure_cross_current(trs, path)`	Card-to-Gamry |H| and phase per card (columns), one line per current.
`build(run_dirs, out_dir, log)`	The four plots per run, the cross-current figure and the tables.
`main(argv)`	—

checks/card_gain.py

670 lines · 2 classes · 18 functions

Purpose. Compares every card's UC2 cell-voltage phasors with the median card to measure each card's voltage-chain gain (and high-frequency response); also card impedance factors and plate-vs-bench current.

Why it exists. Each card divides by its own voltage channel, so a card reading 4 % high puts +4 % on every segment it carries.

Inputs → outputs. Card files + schedule → CardReference objects, applied gains.

Depends on	eis_local, r2d2_geometry, utils
Used by	Local EIS Pipeline Runner, bronze, dc_closure, frequency_response, main, plausibility, silver (+ 2 test files)
Libraries	csv, dataclasses, numpy, pathlib, re, warnings
Python features	@property, NaN handling, boolean masks / np.where, classes, closures / nested functions, complex numbers, comprehensions, context managers (with), csv module, dataclasses, f-strings, generator expressions, lambda, np.errstate, pathlib, regular expressions, try/except, type hints

Class / function	What it does (first docstring line)
`class CardReference @dataclass`	Methods: gain_pct, flat_pct, row.
`class CardFactor @dataclass`	Methods: spread_pct, level_pct, row.
`_opt(cfg, name, default)`	—
`mode(cfg)`	—
`estimate_snr_db(snr_sample_db, n)`	SNR of a fitted phasor from the per-sample SNR of its window: the amplitude of an n-sample least-squares fit is known sqrt(n/2) times better than one sample.
`short(card)`	—
`_band(freq, vals, lo, hi)`	—
`compare_phasors(freq, A, snr, dc, cfg, channel, used_for_Z)`	Card against card for ONE channel name.
`hf_response(freq, A, snr, cfg, ref_card)`	Each card's UC magnitude against ONE reference card, step by step.
`applied_hf(refs)`	card -> per-step |UC| response its Z is divided by (cfg.uc_hf_equalise).
`correctable(ref)`	—
`measure_reference(files, cards, schedule, cfg, lags, log)`	{channel: {card: CardReference}} for every UC channel the cards share.
`load_reference(run_dir)`	Read bronze/card_reference.csv back ({} when the run has none).
`applied_gains(refs)`	card -> what its Z is divided by (1.0 where nothing is applied).
`_log(out, m, log)`	—
`_fkey(f)`	—
`impedance_factors(spectra, card_of, voltage, cfg)`	Per card: how far its segments' |Z| sits from the plate median, per band.
`current_of(condition)`	"450A" -> 450.0; nan when the name carries no current.
`dc_closure_table(rows)`	Fit I_meas = a * I_ref + b over the conditions.
`plate_current(j_dc, areas, plate_key)`	(I over the measured area, measured area, I scaled to the plate).

checks/dc_closure.py

281 lines · 0 classes · 11 functions

Purpose. Sum of segment currents against the bench current over all conditions, fitted as I_plate = a·I_bench + b; also the voltage-sense offset.

Why it exists. a − 1 is a current-scale error that sits in every impedance as well; b is a zero offset that does not.

Inputs → outputs. Run folders (+ MF4/Gamry) → fit result, plot, printed report.

Depends on	card_gain, gamry_compare, gamry_sync, r2d2_geometry, utils
Used by	Local EIS Pipeline Runner, main (+ 1 test files)
Libraries	argparse, csv, json, matplotlib, numpy, pathlib
Python features	JSON, NaN handling, argparse CLI, comprehensions, context managers (with), csv module, f-strings, generator expressions, if __name__ == "__main__", matplotlib, pathlib, raising exceptions, try/except, type hints

Class / function	What it does (first docstring line)
`_rows(path)`	—
`_f(x)`	—
`find_runs(paths)`	—
`bench_current(run, bench_log, gamry_dir, cfg)`	(current, source) for this run's condition.
`uc2_dc(run)`	Median over the cards of the UC2 DC level (bronze/card_reference.csv).
`gamry_vdc(gamry_dir, condition)`	—
`sense_offset(rows, plate_key)`	Fit UC2 - Vdc = b - I * R_x over the conditions.
`analyse(runs, bench_log, gamry_dir, plate_key)`	—
`plot(res, path)`	—
`print_sense(res)`	—
`main(argv)`	—

checks/frequency_response.py

477 lines · 0 classes · 16 functions

Purpose. Measuring-chain frequency response ex-situ (Abgleich bode sweeps / gain file) and in-situ (each segment against the plate median, before and after timing correction), with checks and plots.

Why it exists. Whatever the chain does to either signal ends up in Z, most strongly at the top of the band.

Inputs → outputs. Run folder + gain file or bode folder → frequency_response/ tables, plots, check verdicts.

Depends on	card_gain, gamry_dta, plausibility, utils
Used by	Local EIS Pipeline Runner, main (+ 1 test files)
Libraries	argparse, json, matplotlib, numpy, pandas, pathlib
Python features	JSON, NaN handling, argparse CLI, complex numbers, comprehensions, context managers (with), f-strings, generator expressions, if __name__ == "__main__", lambda, matplotlib, np.errstate, pandas, pathlib, raising exceptions, try/except, type hints

Class / function	What it does (first docstring line)
`_short(card)`	—
`exsitu_from_gain(path)`	G_d per segment from a pipeline gain file (gain = 1/H -> H = 1/gain).
`exsitu_from_bode(folder, f_ref_hz)`	G_d per segment straight from the Abgleich bode/ sweeps.
`_empty_exsitu()`	—

Class / function	What it does (first docstring line)
`_at(df, f0)`	Each segment's point nearest to f0 (log distance).
`exsitu_summary(df, f_top)`	—
`_relative(df, re, im, scale)`	—
`insitu(run_dir)`	Z_s / plate median per segment, raw (bronze) and clean (silver), on the points silver kept.
`insitu_summary(df)`	—
`card_residual(df)`	Per card, after correction: median phase above F_HF, and how far |H| above F_HF moves from the same segment's own 30-1000 Hz level (a flat scale factor does not count, a frequency-dependent card response does).
`checks(summary)`	—
`plot_exsitu(df, path, title, f_top)`	—
`plot_insitu(df, path, title)`	—
`run(run_dir, gain_file, bode_dir, out_dir, write…)`	Both views for one finished run; writes RUN_DIR/frequency_response/.
`report(summary, say)`	—
`main(argv)`	—

checks/gamry_compare.py

1,031 lines · 3 classes · 18 functions

Purpose. Compares the Gamry whole-cell sweep (converted to Ω·cm²) with the local cell aggregate on their shared band, reads the operating point from the MF4 bench log, and writes comparison tables and plots.

Why it exists. The parallel sum of all segments must reproduce an independent whole-cell instrument: the only end-to-end test of calibration, geometry, synchronisation and chain response together.

Inputs → outputs. cell_aggregate.csv + Gamry .DTA (+ MF4) → Comparison objects, gamry_comparison*.csv/png.

Depends on	gamry_dta, utils
Used by	Local EIS Pipeline Runner, csv_pipeline, dc_closure, gamry_sync, main, nyquist, polcurve, silver (+ 4 test files)
Libraries	asammdf, csv, dataclasses, datetime, importlib, matplotlib, numpy, pathlib, re, sys
Python features	@property, NaN handling, classes, closures / nested functions, complex numbers, comprehensions, context managers (with), csv module, dataclasses, f-strings, frozen dataclass, generator expressions, importlib, lambda, matplotlib, pathlib, regular expressions, sys.path, try/except, type hints

Class / function	What it does (first docstring line)
`class CellSweep @dataclass(frozen=True)`	One whole-cell reference spectrum. Methods: name, condition, asr, hfr_ohm.
`class BenchLog @dataclass`	An ASAM MDF4 bench recording, reduced to what the comparison needs. Methods: state_at.
`class Comparison @dataclass`	One local aggregate against one whole-cell sweep. Methods: n_points, hfr_rel, hfr_fit_rel, mag_rel_median, phase_diff_deg, rms_rel, summary.
`_hf_intercept(freq, Z)`	Z' where Z'' crosses zero on the way DOWN from the top of the band.
`_hf_extrapolated(freq, Z, n_top)`	Z' extrapolated to Z'' = 0 from the top of the band.
`read_cell_sweep(path)`	—
`find_cell_sweeps(root, pattern, order_id, version)`	Every whole-cell sweep under `root` that belongs to THIS cell.
`_asammdf_advice(exc)`	Why the import failed, and the command that fixes THIS interpreter.
`_ensure_zstd_importable()`	Make `from zstd import decompress` succeed without a C compiler.

Class / function	What it does (first docstring line)
`read_bench_log(path)`	Read an MF4 bench log. Requires `asammdf`; absent, this is skipped.
`_order_core(text)`	The digits of an order id, so RO2611976-01 and 2611976 compare equal.
`version_of(name)`	The build token in a file name, normalised to "V26_092", or None.
`names_version(name, version)`	Does this filename carry this build token?
`names_order(name, order_id)`	Does this filename name this cell?
`find_bench_log(root, order_id, version)`	The bench .mf4 for this cell, or None.
`compare(freq_local, Z_local, sweep, area_cm2, bench,…)`	Put a local aggregate and a whole-cell sweep on the same axis.
`read_cell_aggregate(path)`	Read the pipeline's own `cell_aggregate.csv` (mohm.cm2) as ohm.cm2.
`write_outputs(comparisons, out_dir, log)`	—
`plot(comparisons, path)`	Nyquist and residual, one column per condition.
`run(results_root, gamry_root, area_cm2, out_dir,…)`	Compare every condition that has BOTH a local result and a sweep.
`_selftest(log)`	Round-trip a known cell through the comparison and check the metrics.

checks/hf_response.py

287 lines · 0 classes · 12 functions

Purpose. Measures the residual high-frequency response of every card and segment across several operating points, validates it leave-one-current-out, and writes a frozen correction file that silver applies.

Why it exists. Acquisition effects are identical at every current while the cell changes, so repeated measurements separate the two.

Inputs → outputs. Several run folders → hf_response file + validation metrics.

Depends on	utils
Used by	Local EIS Pipeline Runner, card_diagnostics (+ 1 test files)
Libraries	argparse, json, numpy, pandas, pathlib
Python features	JSON, argparse CLI, boolean masks / np.where, complex numbers, comprehensions, context managers (with), f-strings, if __name__ == "__main__", lambda, pandas, pathlib, raising exceptions, try/except, type hints

Class / function	What it does (first docstring line)
`load_run(run_dir)`	(condition, spectra with segment / card / fk / Z, config) of one run.
`deviations(sp)`	Each point against the plate median at its frequency (complex).
`_quad(f, y)`	y smoothed by a quadratic in f (least squares): over 2-12.5 kHz the residual phase of every segment is a line in f to 0.9 deg and a quadratic to 0.4 deg. A running median would bend the steep top end.
`_smooth3(y)`	—
`estimate(runs, f_lo)`	{segment: (freq, h)} from deviations() of several runs.
`apply(sp, h)`	Divide each point at or above f_lo by its segment's h (nearest step).
`metrics(sp, f_card)`	Phase spread at the top common step, largest card |Z| step above f_card.
`validate(runs, f_lo)`	Leave one run out: estimate on the others, apply to it, compare.
`write(h, path, f_lo, note)`	The gain-file format utils.load_gain reads; h = 1 below f_lo.
`build(run_dirs, out_dir, f_lo, log)`	Estimate from all runs, validate leave-one-out, write the file.
`_seg_key(s)`	—
`main(argv)`	—

checks/plausibility.py

1,223 lines · 2 classes · 26 functions

Purpose. Plate-level physics checks with PASS/WARN/FAIL/n/a verdicts: area coverage, measured block, current closure, passivity, card alignment, channel lag, card gain, smoothness of R_ohmic, trend along the flow, series-resistance closure, aggregate vs whole cell, chain response.

Why it exists. Point-level gates cannot see a channel map off by one, a wrong area or a swapped inlet; the shape of the finished plate can.

Inputs → outputs. SilverRun (+ Gamry reference) → Report and plausibility.csv.

Depends on	card_gain, plate_stats, r2d2_geometry, silver, utils
Used by	Local EIS Pipeline Runner, frequency_response, main (+ 4 test files)
Libraries	argparse, csv, dataclasses, math, numpy, pathlib, types
Python features	@property, NaN handling, argparse CLI, classes, closures / nested functions, comprehensions, context managers (with), csv module, dataclasses, f-strings, frozen dataclass, generator expressions, if __name__ == "__main__", lambda, pathlib, raising exceptions, try/except, type hints

Class / function	What it does (first docstring line)
`class Check @dataclass(frozen=True)`	One plausibility question, its answer, and why that is the answer.
`class Report @dataclass`	Methods: verdict, by_verdict, rows.
`coverage(measured, plate_key)`	How much of the plate this run actually saw, by area.
`describe_block(measured, plate_key, max_missing)`	Name the measured set: whole plate, interior block, perimeter, or neither.
`_contiguous_runs(nums)`	[1,2,3,7,8] -> [(1,3),(7,8)].
`resolve_current(cfg, bench)`	The plate current to close against, and where it came from.
`current_closure(j_dc, areas, setpoint_a, plate_key, source)`	Sum(j_s * A_s) over the measured area, scaled to the whole plate.
`_positions(plate)`	Segment -> (x, y) mm of its label pad: where the drawing places it.
`_tile_or_all(values, pos)`	The 36 tile segments (plate_stats: 1..36, one per 5x5-pad tile, the edge segments inside the tiles not counted) when at least 8 of them have a value; every segment otherwise.
`_basis(good)`	—
`neighbour_smoothness(values, plate_key, param)`	Is the map spatially smooth, or does it look like shuffled labels?
`passivity(spectra)`	Re(Z) > 0 everywhere: a passive cell cannot generate energy.
`flow_trend(values, plate_key, param, axis)`	Does the map vary along the flow axis at all, and in which direction?
`parallel_series_resistance(r_ohmic, areas, r_sd)`	Combine per-segment R_s (ohm.cm2) in parallel, back into ohm.cm2.
`series_resistance_closure(r_ohmic, areas, reference_hfr_ohm_cm2, r_sd,…)`	Parallel sum of the segment R_s against the whole cell's own R_s.
`_topband_re(freq, Z, span_decades)`	Inverse-variance mean of Re Z over the top third-decade of a band.
`matched_series_resistance(freq, Z_agg, ref_freq, ref_Z_asr, area_fract…)`	R_s of the local aggregate vs the whole cell, SAME estimator, SAME band.
`aggregate_vs_reference(freq, Z_agg, ref_freq, ref_Z_asr, area_fract…)`	The partial-plate aggregate against a whole-cell sweep, both in ohm.cm2.
`channel_lag_check(lags)`	Do the segment current chains see the same high-frequency phase?
`card_alignment_check(card_lags)`	Was every card shifted onto the common time base?
`card_voltage_check(card_ref)`	Do the cards read the same cell voltage on their UC channel?
`card_factor_check(factors)`	Is any card's impedance level a scale error rather than the cell?
`check_run(sr, cfg, plate_key, reference, reference_hfr…)`	Every check that the available data supports, on a finished SilverRun.
`check_from_disk(run_dir, cfg, plate_key, reference, referenc…)`	The same checks as check_run, from files a finished run wrote.

Class / function	What it does (first docstring line)
`report(rep, log)`	—
`save(rep, out_dir)`	—
`_load_summary(path)`	—
`main(argv)`	—

checks/segment_scale.py

224 lines · 0 classes · 8 functions

Purpose. Detects per-segment impedance scale factors that are the same at every frequency and every load.

Why it exists. A constant factor across conditions points to the segment's current path (K), not to the cell. Reported, not applied.

Inputs → outputs. Run folders → scale table, optional gain file.

Depends on	utils
Used by	Local EIS Pipeline Runner (+ 1 test files)
Libraries	argparse, json, numpy, pandas, pathlib
Python features	JSON, argparse CLI, boolean masks / np.where, comprehensions, context managers (with), f-strings, generator expressions, if __name__ == "__main__", lambda, pandas, pathlib, raising exceptions, try/except, type hints

Class / function	What it does (first docstring line)
`_runs(paths)`	—
`_label(run)`	—
`run_factors(run, f_lo, f_hi)`	Per measured segment of one run: log factor over the band and its two halves (natural log of |Z_seg / plate median|).
`analyse(runs, f_lo, f_hi, min_pct, flat_pct, repeat_…)`	—
`gain_rows(scale, chain, f_min, f_max, n)`	Gain-file rows: G = k (times the chain response when given).
`write_gain(path, scale, chain_file, note)`	—
`report(res, log)`	—
`main(argv)`	—

checks/validate_hf.py

197 lines · 0 classes · 7 functions

Purpose. Synthetic self-validation of the high-frequency chain: builds cards with known impedance and multiplexer skew and checks the estimators recover the truth.

Why it exists. Every accuracy claim is tested against known truth with the skew present.

Inputs → outputs. Nothing → pass/fail report (used by main.self_test).

Depends on	config, silver, utils
Used by	main
Libraries	numpy
Python features	NaN handling, complex numbers, comprehensions, f-strings, generator expressions, if __name__ == "__main__", raising exceptions, type hints

Class / function	What it does (first docstring line)
`synth_card(rng, n_seg, n_ch, fs, f_max, noise, ref_slot…)`	One card: n_seg segments, each at its own conversion slot.
`_prep(items)`	Pack the synthetic card the way silver.optimise_skew expects it.

Class / function	What it does (first docstring line)
`_cost(items, dt0, k, M)`	—
`fit_structural(items, k_nom)`	Calls silver's REAL optimiser, so this tests shipped code.
`fit_per_card(items)`	Legacy: one free delay for the whole card.
`report(name, est, true)`	—
`main()`	—

checks/via_resistance.py

347 lines · 0 classes · 11 functions

Purpose. Via/shunt resistance against temperature per segment and card from the Abgleich step files.

Why it exists. Shows how much the via resistance moves with temperature and therefore how much it could shift the local current density.

Inputs → outputs. Abgleich folder (+ run folders) → summary, plots.

Depends on	only standard / third-party libraries
Used by	Local EIS Pipeline Runner (+ 1 test files)
Libraries	argparse, json, matplotlib, numpy, pandas, pathlib, re
Python features	JSON, argparse CLI, comprehensions, f-strings, if __name__ == "__main__", lambda, matplotlib, pandas, pathlib, raising exceptions, regular expressions, try/except, type hints

Class / function	What it does (first docstring line)
`_short(card)`	—
`_pairs(path)`	—
`read_steps(abgleich_dir)`	One row per segment and calibration step: temperature and slope.
`run_context(run_dirs)`	card per segment, and {condition: {segment: T_degC}} from finished runs (bronze/segment_meta.csv).
`analyse(steps, curr_csv, card, temps)`	Per segment: level, temperature coefficient, hysteresis, and k at the operating temperature of every run given.
`summary(seg, temps)`	—
`report(s, say)`	—
`plot_vs_temperature(steps, seg, path, temps, title)`	—
`plot_alpha(seg, path, title)`	—
`run(abgleich_dir, run_dirs, out_dir, write_png, …)`	—
`main(argv)`	—

6.5 analysis/

Optional scientific analyses on top of the results: a DRT-informed circuit fit and polarisation curves.

analysis/ecm_drt.py

354 lines · 0 classes · 12 functions

Purpose. DRT-informed equivalent-circuit fit (method B): arcs sorted by τ, weighting by the propagated uncertainty, optional transmission-line element for the catalyst layer, masking of mains-frequency points.

Why it exists. A more careful circuit fit for the thesis, used by the extra-analyses notebook.

Inputs → outputs. freq, Z, sigma (+ DRT peaks) → fit dict, model curves, summaries.

Depends on	only standard / third-party libraries

Used by	Local EIS Extra Analyses, Local EIS Pipeline Runner (+ 1 test files)
Libraries	numpy, scipy
Python features	NaN handling, SciPy, boolean masks / np.where, closures / nested functions, complex numbers, comprehensions, f-strings, generator expressions, lambda, try/except, type hints

Class / function	What it does (first docstring line)
`z_zarc(w, R, tau, n)`	—
`z_tlm(w, R_ion, R, tau, n)`	Finite catalyst-layer transmission line with a ZARC interface.
`_unpack(p, layout)`	layout = ('zarc',) | ('zarc','zarc') | ('tlm','zarc'); p after Rs, L.
`z_model(Rs, L, p_arcs, layout, freq)`	—
`mains_mask(freq, mains_hz, odd_only, tol, max_order)`	True where a point sits on a low mains harmonic (to be dropped).
`_start(freq, Z, Rs, n_arcs, peaks)`	—
`_fit_layout(f, Z, s, layout, Rs0, rs_fixed, peaks, tau_l…)`	—
`fit_drt_ecm(freq, Z, sigma_rel, peaks, rs_fixed, f_max_f…)`	Fit one spectrum. Returns a dict (see module docstring).
`_report(b, f, Zf, s, weight, dropped_mains, rs_fixed…)`	—
`model_curve(fit, freq)`	The fitted model on any frequency grid (e.g. a smooth line).
`summarize(fits)`	{key: (median, q25, q75, n)} over ALL fitted segments + counts.
`arcs_from_params(params, n_arcs)`	Method A (csv_pipeline) params -> the same fast/slow naming.

analysis/polcurve.py

349 lines · 1 classes · 7 functions

Purpose. Polarisation curves (voltage against current density) from FAMOS (whole plate and three flow bands) and from Gamry sweeps, drawn in the bench's style.

Why it exists. A segmented plate gives local polarisation curves that a whole-cell curve cannot show.

Inputs → outputs. Run folders / .DTA files → PolCurve objects, figures.

Depends on	config, eis_local, gamry_compare
Used by	Local EIS Pipeline Runner (+ 1 test files)
Libraries	csv, dataclasses, json, matplotlib, numpy, pandas, pathlib, re
Python features	JSON, NaN handling, classes, comprehensions, csv module, dataclasses, f-strings, generator expressions, lambda, matplotlib, pandas, pathlib, raising exceptions, regular expressions, try/except, type hints

Class / function	What it does (first docstring line)
`class PolCurve @dataclass`	Methods: sorted, ir_free.
`plot_polcurves(curves, title, ylabel, xlabel, xlim, ylim, l…)`	The bench comparison style. Returns the figure.
`read_gamry_dc(path)`	Vdc, Idc and HFR of one Gamry EIS sweep (.dta).
`gamry_polcurve(dta_files, label, area_cm2, n_cells, color)`	One point per sweep -> (PolCurve, table rows).
`famos_cell_voltage(files, ref_channel, stride)`	DC level of the cell-voltage channel on every card -> median, spread.
`famos_current_density(out_dir)`	Plate and flow-band current density from a run's gold outputs.
`famos_polcurves(runs, ref_channel, label, bands, v_override)`	runs: condition -> (list of FAMOS files, run out_dir).
`load_curve_csv(path, label, j_col, v_col, dashed)`	A curve from a CSV (e.g. a bench export or a reference cell).

6.6 plotting/

Everything that draws: Nyquist/Bode figures, plate heat maps (static and interactive), the drop-down viewers and the common style.

plotting/bench_plots.py

325 lines · 1 classes · 7 functions

Purpose. Test-bench MF4 channels as one interactive Plotly figure (parameter drop-down, legend toggling, unified hover, range slider).

Why it exists. Lets the user check the operating point at the time of each sweep.

Inputs → outputs. MF4 file (asammdf) → Channel list → Plotly figure.

Depends on	only standard / third-party libraries
Used by	Local EIS Pipeline Runner (+ 1 test files)
Libraries	asammdf, dataclasses, numpy, plotly
Python features	@property, NaN handling, classes, closures / nested functions, comprehensions, dataclasses, f-strings, generator expressions, lambda, plotly, try/except, type hints

Class / function	What it does (first docstring line)
`class Channel @dataclass`	Methods: stats.
`decimate(t, v, max_points)`	At most ~max_points samples, keeping each bucket's min AND max so a spike survives the thinning (a plain stride would drop it).
`_clean(t, v)`	—
`channels_from_mdf(mdf, groups, include_others, max_points)`	Every known channel present in an open asammdf.MDF, then (optionally) every other numeric channel of the file under OTHER_GROUP.
`read_mf4(path, include_others, max_points)`	Read an MF4 bench log. Requires `asammdf`.
`mf4_start_time(path)`	The recording's start time (datetime) or None.
`_fmt(x)`	—
`bench_figure(channels, title, start, reference_lines, hei…)`	One figure, every channel a trace; a dropdown chooses what is shown.

plotting/figure_panels.py

217 lines · 0 classes · 6 functions

Purpose. Splits a multi-panel Plotly figure (make_subplots) into separate full-width figures, keeping traces, axes, shapes and legends per panel.

Why it exists. Side-by-side panels squeeze the Nyquist arc; full-width panels are readable without rewriting every cell.

Inputs → outputs. Plotly figure → list of Plotly figures.

Depends on	only standard / third-party libraries
Used by	Local EIS Pipeline Runner, viewers (+ 1 test files)
Libraries	copy, plotly, re
Python features	closures / nested functions, comprehensions, f-strings, generator expressions, lambda, plotly, regular expressions, try/except, type hints

Class / function	What it does (first docstring line)
`_panels(layout)`	(xref, yref, xkey, ykey) for every x axis anchored to a y axis.
`_clean_axis(ax, own_ref, new_ref)`	—
`_remap_ref(ref, xref, yref)`	'x3' -> 'x', 'x3 domain' -> 'x domain'; None when it is another panel's.
`_title_text(layout)`	—
`_with_panel(title, sub)`	—

Class / function	What it does (first docstring line)
`split_subplots(fig, panel_height, panel_width, skip_empty)`	One figure per subplot panel, in row-major order. See module docstring.

plotting/nyquist.py

462 lines · 0 classes · 13 functions

Purpose. The per-segment Nyquist/Bode figure, coloured by position along the air path, with the Gamry overlay; saves the same figure as PNG/HTML; also the aggregate-vs-Gamry comparison figure.

Why it exists. What is saved is exactly what was looked at; colour by flow position makes gradients visible.

Inputs → outputs. spectra_clean.csv / DataFrames → Plotly figures, PNG, HTML.

Depends on	gamry_compare, plate_style, r2d2_geometry
Used by	Local EIS Pipeline Runner, card_diagnostics, csv_pipeline, gold (+ 2 test files)
Libraries	matplotlib, numpy, pandas, pathlib, plotly
Python features	complex numbers, comprehensions, f-strings, generator expressions, matplotlib, pandas, pathlib, plotly, try/except, type hints

Class / function	What it does (first docstring line)
`gamry_frame(src, area_cm2)`	The whole-cell Gamry sweep as a DataFrame (freq_hz, z_re_mohm_cm2, z_im_mohm_cm2), every point exactly as in the file, ascending f.
`find_gamry(gamry_dir, condition, order_id, version, are…)`	gamry_frame of the condition's sweep in `gamry_dir` (the same lookup the pipeline uses: this order's build only), or None.
`_add_gamry(fig, g, name, width)`	The Gamry, black, in all three panels; one legend entry.
`air_inlet_x()`	x (plate mm) of the cathode (air) inlet end.
`flow_position(segments)`	0 at the cathode inlet .. 1 at the cathode outlet, per segment.
`_jet(t)`	—
`colours(segments, colour)`	—
`_seg_key(s)`	—
`figure(df, title, colour, rebuilt, note, height, wi…)`	Nyquist + |Z| + phase of every segment (one subplot each).
`save_png(df, path, title, colour, rebuilt, dpi, gamry…)`	The Nyquist panel of `figure`, as a PNG: same data, same colours.
`save_run(run_dir, title, colour, out_dir, gamry)`	nyquist.png + nyquist.html for one finished run (FAMOS or CSV).
`comparison_figure(aggregate, gamry, title, aggregate_name, gam…)`	The aggregate of the segments against the whole-cell Gamry: Nyquist, |Z| and phase, those two curves and nothing else.
`comparison_png(aggregate, gamry, path, title, dpi)`	`comparison_figure` as a PNG (Nyquist, |Z|, phase in one row).

plotting/plate_figure.py

522 lines · 0 classes · 12 functions

Purpose. Static plate heat maps in the plate-viewer style (frame, ports, sensors), per segment or 2D interpolated, and the per-condition map writer.

Why it exists. Publication-style maps for reports and the gold folder.

Inputs → outputs. {segment: value} → matplotlib figures / PNG files.

Depends on	config, plate_stats, plate_style, r2d2_geometry
Used by	Local EIS Pipeline Runner, csv_pipeline, gold, viewers (+ 3 test files)
Libraries	csv, matplotlib, numpy, pathlib, scipy, sys

Python features	SciPy, closures / nested functions, comprehensions, csv module, f-strings, generator expressions, if __name__ == "__main__", lambda, matplotlib, module __getattr__, pathlib, raising exceptions, try/except, type hints

Class / function	What it does (first docstring line)
`_cmap(name)`	—
`_limits(param, vals, limits)`	—
`draw_flow_plate(values, param, title, subtitle, classes, lim…)`	One plate heat map in the viewer's layout. Returns the figure.
`node_xy(p, n)`	Where segment n sits on the maps: the centre of its label pad, the red number box of the plate drawing (r2d2_geometry.label_xy).
`interpolate_field(p, vals, W, H, step)`	The value field over the active area: 2D linear interpolation between the segments' label pads (the red number boxes of the drawing, so every value sits exactly where the drawing places its segment), an inverse-distance
`shaded_rgb(gx, gy, zi, cm, norm, W, H, gloss)`	Colour the field and give it the soft gloss: a light relief from the field itself (light from the top left) and a faint diagonal sheen.
`_draw_interpolated(ax, p, vals, classes, cm, norm, W, H, gloss)`	Fill the active area with the interpolated field and mark every segment with a square.
`_fixed(param)`	—
`renders(style_)`	The map styles a heatmap_style setting asks for.
`suffix(render)`	File-name suffix: plate_R_ohmic.png / plate_R_ohmic_interp.png.
`__getattr__(name)`	—
`write_condition_maps(summary_csv, out_dir, title, params, dpi, si…)`	Draw plate_<param>.png for every param from a gold plate_summary.csv (render "interpolated": plate_<param>_interp.png; "both": both).

plotting/plate_maps.py

224 lines · 0 classes · 4 functions

Purpose. Static maps with the value written inside every segment and robust colour limits.

Why it exists. A narrow-range map (like R_ohmic) is unreadable when one outlier stretches the colour bar.

Inputs → outputs. {segment: value} → matplotlib figure.

Depends on	plate_stats, plate_style, r2d2_geometry
Used by	Local EIS Pipeline Runner (+ 2 test files)
Libraries	matplotlib, numpy
Python features	comprehensions, f-strings, generator expressions, matplotlib, try/except, type hints

Class / function	What it does (first docstring line)
`_finite(values)`	—
`uniformity_stats(values, classes)`	n, mean, median, sd, CV [%], min, max, span relative to the mean [%], over the 36 representative segments (1..36, one per 5x5-pad tile; the edge segments 37..72 inside those tiles are not counted) -- plate_stats.
`robust_limits(values, pct)`	Colour limits from percentiles, widened so a flat plate is not noise.
`draw_value_map(values, label, unit, cmap, decimals, title, …)`	Plate map, one colour and one printed value per segment.

plotting/plate_plotly.py

284 lines · 1 classes · 9 functions

Purpose. Interactive Plotly plate map on the 45×20 pad grid: true segment shapes, value printed, hover, parameter drop-down.

Why it exists. Readable the same way as the static maps but interactive in the notebook.

Inputs → outputs. Field objects → Plotly figure.

Depends on	config, plate_stats, plate_style, r2d2_geometry
Used by	Local EIS Pipeline Runner, gold, viewers (+ 1 test files)
Libraries	dataclasses, numpy, plotly
Python features	NaN handling, classes, closures / nested functions, comprehensions, dataclasses, f-strings, generator expressions, plotly, try/except, type hints

Class / function	What it does (first docstring line)
`class Field @dataclass`	One parameter on the plate. `values`: segment -> value, display units.
`_clean(values)`	—
`_limits(fd, vals)`	—
`_rgba(colorscale, t)`	Colour of the scale at t in [0, 1] as an RGB(A) tuple in 0..1.
`_boundary_xy(p)`	Line segments between pads owned by different segments, + outline.
`_label_xy(p, n, s)`	—
`_field_traces(fd, p, show_colorbar)`	Heatmap + number labels + value labels for one field.
`_subtitle(fd)`	Counts, then mean / median / sd over the 36 tile segments (plate_stats: segments 1..36, one per 5x5-pad tile).
`_end_annotations(W, H)`	Inlet / outlet / coolant labels beside the two ends, with arrows pointing the way each stream goes ON SCREEN.
`interactive_plate(fields, title, subtitle, plate_name, height,…)`	One Plotly figure; a dropdown switches between `fields`.

plotting/plate_style.py

265 lines · 0 classes · 16 functions

Purpose. One look for every plate map: Jet colour scale (matplotlib and Plotly), text sizes, view orientation, gas/coolant port labels and positions.

Why it exists. The plate is drawn in six places; one style module guarantees one colour means one value and top-left means the same corner everywhere.

Inputs → outputs. config settings → colour maps, labels, coordinates.

Depends on	config
Used by	Local EIS Pipeline Runner, nyquist, plate_figure, plate_maps, plate_plotly, r2d2_geometry, viewers (+ 1 test files)
Libraries	matplotlib
Python features	comprehensions, f-strings, generator expressions, matplotlib, try/except, type hints

Class / function	What it does (first docstring line)
`_rgb01(spec)`	—
`mpl_cmap(name)`	Matplotlib colormap for the heat maps (plotly Jet unless overridden).
`plotly_colorscale(name)`	The same scale as a Plotly colorscale (Jet; "parula" on request).
`ink(rgba)`	Text colour that stays legible on a fill (WCAG relative luminance).
`mirrored()`	True when the plate is drawn mirrored (read at call time, so a notebook can flip config.PLATE_VIEW_MIRRORED and redraw).
`xlim(lo, hi)`	Matplotlib x limits: reversed when mirrored, so every patch, line and text position mirrors with the axis and nothing has to be recomputed.
`outward_ha(physical_side)`	Horizontal alignment for text set just OUTSIDE the x = 0 ("x0") or x = W ("xW") edge, so it grows away from the plate in either view.
`screen_side(physical_side)`	'left' or 'right': where the x = 0 / x = W edge lands on screen.
`coolant_inlet_end()`	'x0' or 'xW': the plate end the coolant enters at (config).
`flow_arrangement()`	—
`end_streams()`	Per plate end ('x0', 'xW'): (label, role, colour) of what enters or leaves there, gas ports first, coolant last.

Class / function	What it does (first docstring line)
`port_slot(label)`	Where a port sits in its end's column: the coolant in the middle, the gas outlet above it and the gas inlet below it.
`port_kind(label)`	'anode', 'cathode' or 'coolant'.
`_arrow(stream_from)`	'left to right' / 'right to left' for a stream entering at an end.
`flow_note()`	One line describing the flows AS DRAWN (it changes with mirroring).
`annotate_ends(ax, width_mm, height_mm, gap_mm, fontsize)`	Write what enters and leaves at each end beside a plain plate map.

plotting/viewers.py

589 lines · 0 classes · 16 functions

Purpose. Builds ONE HTML block with drop-downs (condition, parameter, view …) and a single plot area from many Plotly figures or HTML items; also plate-map viewers and table/image helpers.

Why it exists. Stacked figures fill the notebook; one plot with selectors is readable and still works when saved as a file.

Inputs → outputs. {(choice, …): figure} → HTML string for displayHTML.

Depends on	figure_panels, plate_figure, plate_plotly, plate_style, r2d2_geometry
Used by	Local EIS Pipeline Runner (+ 5 test files)
Libraries	base64, gzip, html, io, json, matplotlib, numpy, pandas, pathlib, plotly, re, uuid
Python features	JSON, closures / nested functions, comprehensions, f-strings, generator expressions, pandas, pathlib, raising exceptions, try/except, type hints

Class / function	What it does (first docstring line)
`_plotlyjs_url()`	—
`_slim(obj, sig)`	Plotly writes numeric arrays as binary blocks ({"dtype", "bdata"}), which do not compress. Turn them into plain numbers rounded to `sig` significant digits (far below what a plot can show) so gzip can work.
`_payload(item)`	A figure as {"fig": json} or HTML as {"html": str}.
`_pack(obj)`	JSON -> gzip -> base64: the payload of a page. Plot data compresses 5-10x, and Databricks drops an HTML output that is too large.
`_shell(dims, opts, payload, render_js, title, note,…)`	Drop-downs + one plot area; the browser unpacks the payload and calls render(key, P, plot, box) on every change. `render_js` defines render.
`selector_html(items, dims, title, height, note)`	One drop-down per entry of `dims`, one plot area.
`panels(fig, panel_height, panel_width)`	A (multi-panel) figure as [(panel name, single-panel figure)].
`by_condition(figs, title, note, panel_dim)`	{condition: figure or html} -> selector; a multi-panel figure becomes a second drop-down (`panel_dim`) over its panels.
`img_html(path, width)`	A PNG as an inline <img>, for selectors over saved figures.
`table_html(df, decimals)`	—
`plate_fields(summary_csv, fields)`	plate_plotly.Field per parameter present in a plate_summary.csv.
`_plate_geometry(p, gloss)`	Everything every map of the plate shares, sent once: the pad owner grid, the segment outlines, label points, centres, the inlet / outlet labels, the two colour scales and the gloss overlay.
`_plate_item(p, fd, head, step)`	One condition x parameter: the values, and the interpolated field on a 4 mm grid quantised to 0..250 and stored as differences along each row (about 2 kB once compressed; the browser smooths it, so the grid does not show
`plate_maps(summaries, title, fields, views, gloss)`	{condition: plate_summary.csv or list[Field]} -> one heat-map viewer with Condition / Parameter / View drop-downs.
`save_page(fragment, path, title)`	A viewer (what displayHTML shows) as a standalone .html file: the same drop-downs and plots, opened in any browser. Plotly comes from the CDN.
`unpack_payload(html)`	The selector's gzip+base64 payload, decoded (tests only).

6.7 readers/

File readers for everything that is not a FAMOS recording: Gamry .DTA, the Abgleich calibration files, CSV logger files, plus a compatibility shim.

readers/_vendor/zstd.py

47 lines · 0 classes · 0 functions

Purpose. Stand-in for the PyPI package zstd, which asammdf imports but which has no Windows wheels for Python ≥ 3.11; forwards to zstandard.

Why it exists. Lets the MF4 bench log be read on a measurement PC without a C compiler.

Inputs → outputs. Import of zstd → zstandard.compress/decompress.

Depends on	only standard / third-party libraries
Used by	—
Libraries	zstandard
Python features	—

readers/abgleich.py

324 lines · 1 classes · 7 functions

Purpose. Reads the raw Step<k>_<T>Grad.csv calibration files and rebuilds/verifies curr.csv and temp.csv from them.

Why it exists. A delivered calibration can be checked rather than trusted.

Inputs → outputs. Abgleich folder → fitted coefficients, verification report, coefficient files.

Depends on	eis_local
Used by	Local EIS Pipeline Runner
Libraries	argparse, dataclasses, json, numpy, pathlib, re
Python features	@property, JSON, NaN handling, argparse CLI, classes, comprehensions, context managers (with), dataclasses, f-strings, frozen dataclass, generator expressions, if __name__ == "__main__", lambda, linear algebra (np.linalg), pathlib, raising exceptions, regular expressions, try/except, type hints

Class / function	What it does (first docstring line)
`class StepFile @dataclass(frozen=True)`	Methods: segments.
`read_step(path)`	Parse one Step<k>_<T>Grad.csv.
`read_campaign(folder)`	Every Step file in a folder, ordered by its index.
`_slope(points)`	Least-squares u = R*i + u0 over one segment's DC sweep.
`fit_coefficients(steps, area_cm2)`	Rebuild the curr.csv / temp.csv coefficients from the Step files.
`implied_area(steps, curr_csv)`	The constant R(T)/K(T) a delivered curr.csv implies, per segment.
`verify(folder, curr_csv, temp_csv)`	Read a calibration folder and report whether it hangs together.
`write_coefficients(fit, curr_path, temp_path)`	—

readers/csv_source.py

1,405 lines · 1 classes · 35 functions

Purpose. Reader for every CSV layout (R2-D2 logger sweep, records, wide/long time series, ready spectra, Gamry folders) into one CsvMeasurement shape, with dialect detection, timestamp regularity, quantisation step and non-uniform phasor fits.

Why it exists. The CSV path needs one input shape whatever the file looked like; a reader that half-succeeds on the wrong layout is refused rather than tolerated.

Inputs → outputs. CSV file/folder → CsvMeasurement.

Depends on	gamry_dta, r2d2_geometry, utils

Used by	Local EIS Pipeline Runner, csv_pipeline, main (+ 2 test files)
Libraries	dataclasses, inspect, io, json, numpy, pathlib, re, sys, tempfile
Python features	@property, JSON, NaN handling, boolean masks / np.where, classes, closures / nested functions, complex numbers, comprehensions, context managers (with), dataclasses, f-strings, generator expressions, if __name__ == "__main__", lambda, linear algebra (np.linalg), np.errstate, pathlib, raising exceptions, regular expressions, try/except, type hints

Class / function	What it does (first docstring line)
`class CsvMeasurement @dataclass`	Whatever the CSV held, in the one shape csv_pipeline understands. Methods: segments, n_samples, fs_nominal, duration_s, summary.
`regularity(t)`	How uniform is the recorded time base?
`quantisation_step(x)`	Recover the printed resolution of a column from the data itself.
`fit_phasor_nonuniform(t, y, f, detrend)`	Least-squares sine fit at known f on ARBITRARY sample times.
`fit_phasor(t, y, f, uniform, fs, detrend)`	Dispatch to the uniform-grid fit when that is safe, else to the timestamp fit. Same signature and convention either way.
`fit_phasors_multi(t, y, freqs, detrend)`	Fit EVERY tone of a multisine in one least-squares solve.
`_sniff(path, n_lines)`	—
`_split(line)`	Split a header/data row on the separator that yields the most fields.
`detect_dialect(path)`	Decide which layout `path` is, from its first lines.
`read_records(path)`	The bench record format: one `s<n>: ...` record per segment per instant.
`_read_table(path)`	Header, numeric block and the raw string rows (for non-numeric ids).
`read_wide_time(path)`	time_s, u_cell, s1..s72 [, temp1..temp4] -- one row per instant.
`read_long_time(path)`	time_s, segment, u_s [, u_cell] -- tidy, one row per sample per segment.
`read_freq(path, z_unit)`	A ready-made spectrum table: segment, freq_hz, and Z in some form.
`_r2d2_seconds(stamps)`	Seconds from the first row, from `YYYY.MM.DD HH:MM:SS,ffffff`.
`read_r2d2(path, meta_extra)`	One R2-D2 point file: `<name>.csv` with the `timeshifts` header row.
`read_r2d2_metadata(folder)`	The free-text `metadata.csv` sidecar, as key/value where it looks it.
`r2d2_point_files(folder)`	The point files of a sweep, in acquisition order.
`read_r2d2_sweep(folder)`	Every point file of one sweep, each carrying the shared metadata.
`read_gamry(path, area_cm2)`	A folder of per-segment Gamry .DTA sweeps, or one file.
`is_spectra_dir(folder)`	A folder of z<n>.csv spectra files (the bench evaluation tool).
`operating_point(folder)`	Temperature and current from a folder name like Spectrum2_65degC_449A.
`_spectrum_lines(text)`	—
`read_spectra_dir(folder)`	Every z<n>.csv of one operating point, in mOhm*cm2, with the segment positions and temperatures from their headers, the cell spectrum and the tool's HFR table in `meta`.
`mirror_map_x()`	Segment -> the segment at the mirrored x (x -> plate width - x, same y), on the current plate.
`mirror_x(m)`	Re-assign every segment of a spectra_dir to its left-right mirror.
`spectra_dirs(root)`	Every spectra_dir folder at or below `root` (one level of nesting).
`condition_current(condition)`	—
`match_condition(folders, condition, tol)`	The folder whose current is nearest to `condition` (e.g. "450A" -> Spectrum2_65degC_449A), within `tol` relative; None if there is none.
`is_condition_folder(folder)`	One operating point: a spectra_dir, or a sweep folder whose name carries the current (Spectrum10_65degC_450A).
`condition_folders(root, depth)`	Every operating-point folder at or below `root`, `depth` levels deep.
`campaigns(root, depth)`	{campaign label: [operating-point folders]}. The campaign is the folder that HOLDS the operating points, labelled relative to `root` ("." when the points sit directly in it).

Class / function	What it does (first docstring line)
`nominal_label(current_a)`	449 A -> "450A": the setpoint, when the measured current is within 2 % of a multiple of 5 A; the rounded current otherwise.
`condition_map(folders)`	{"450A": folder, ...} for the folders that carry a current. Two folders with the same setpoint: the one closer to it, then the newer one, wins (the loser is named in `condition_map.duplicates`).
`read(path, dialect)`	Read a CSV measurement, detecting the layout unless told which.
`_selftest(log)`	Round-trip a known spectrum through each layout. Returns #failures.

readers/gamry_dta.py

477 lines · 1 classes · 11 functions

Purpose. Reads Gamry .DTA files (ZCURVE table) and turns the Abgleich bode sweeps into the chain gain file. Cross-checks the bode index against curr.csv and refuses an inconsistent pairing; writes gain = 1/H because the pipeline divides Z by the gain.

Why it exists. The measuring chain rolls off (−2.5° at 1 kHz, −11° at 4.5 kHz); without the gain file that error stays in every Z.

Inputs → outputs. .DTA files → GamrySweep objects, chain_gain CSV.

Depends on	channel_lag, eis_local
Used by	Local EIS Extra Analyses, Local EIS Pipeline Runner, csv_source, frequency_response, gamry_compare (+ 2 test files)
Libraries	argparse, dataclasses, numpy, pathlib, re
Python features	@property, argparse CLI, classes, complex numbers, comprehensions, context managers (with), dataclasses, f-strings, frozen dataclass, generator expressions, if __name__ == "__main__", lambda, pathlib, raising exceptions, regular expressions, try/except, type hints

Class / function	What it does (first docstring line)
`class GamrySweep @dataclass(frozen=True)`	Methods: sorted, z_lf, z_flat, normalised.
`_f(s)`	—
`read_dta(path)`	Parse the ZCURVE table of a Gamry .DTA file.
`read_bode_folder(folder, skip_raw)`	Read every per-segment sweep in a `bode/` folder -> {segment: sweep}.
`find_abgleich_dirs(roots, max_depth)`	Folders that look like an Abgleich delivery: a `bode/` holding per-segment sweeps (*_#<n>.DTA) next to `coefficients/curr.csv`.
`cross_check_abgleich(sweeps, curr_csv)`	Do the bode sweeps and the DC Abgleich describe the same segments?
`_normalised(sw, f_ref_hz)`	—
`median_response(sweeps, f_ref_hz)`	The plate's shared chain response: median |H| and median arg H.
`write_gain_csv(sweeps, path, convention, f_ref_hz, curr_csv…)`	Write `segment,freq_hz,gain_real,gain_imag` for utils.load_gain.
`chain_tau(sweeps, f_lo, f_hi)`	Per segment: the first-order time constant tau [s] of its chain, from arg H = atan(w tau) over f_lo..f_hi -- the same model and the same sign as channel_lag.py fits in situ (Z_meas = Z_true / H, H = 1 + j w tau), so the
`compare_chain_tau(bode_tau, insitu_tau)`	Does the ex-situ chain explain the in-situ per-segment lag?
`chain_summary(sweeps, at_hz)`	Median |H| and phase across segments at a few frequencies.

6.8 docs/

Documentation, plus one small script that generates a measurement test plan.

docs/test_plans/make_test_plan.py

192 lines · 0 classes · 6 functions

Purpose. Generates a five-test measurement plan (Gamry + FAMOS) for the gen1 plate from the bench operating table.

Why it exists. Planning help for future measurements; not part of the evaluation.

Inputs → outputs. Constants → future_test_plan.csv, future_test_plan_overview.csv.

Depends on	only standard / third-party libraries
Used by	—
Libraries	csv, math, pathlib
Python features	comprehensions, context managers (with), csv module, f-strings, pathlib

Class / function	What it does (first docstring line)
`psat(t)`	—
`dpt_for_rh(rh, t_ref)`	—
`rh(dpt, t_ref)`	—
`iac(i)`	—
`row(tid, name, step, phase, i, direction, hold, …)`	—
`add(test)`	—

6.9 tests/

Run with `python -m pytest tests -q`. Each test file plants a specific defect (a lag, a scale error, a swapped channel, a wrong file name) in synthetic data and asserts that the code which should catch it does. The generators and `conftest.py` are described first.

tests/conftest.py

Purpose. Imported automatically by pytest before any test file; imports eis_paths so tests can import modules by plain name.

Why. One place to set up the import path for ~450 tests.

tests/make_synth_famos.py

Purpose. Writes synthetic FAMOS card files with known impedance, a per-channel multiplexer skew and an open high-frequency arc.

Why. Tests can check that the pipeline recovers a truth it was not told.

tests/make_synth_sweep.py

Purpose. Writes a descending stepped sweep on several cards (each armed at a different moment, each channel one ADC slot later) plus the matching Gamry .DTA and calibration files.

Why. End-to-end tests, including the Gamry synchronisation, run on data with known truth.

Test file	Tests	What it checks (module docstring)
`test_aggregate_and_fill.py`	14	The cell aggregate, written as an admittance sum, and the gaps in it.
`test_alignment_diagnostics.py`	14	The alignment band, the clock-drift diagnostic and the closure check.
`test_audit_corrections.py`	5	The corrections that came out of the raw-signal audit of 2612030 450 A:
`test_bench_plots.py`	4	The MF4 bench parameters: one figure, a dropdown per parameter, legend

Test file	Tests	What it checks (module docstring)
test_card_alignment.py	6	Cross-card alignment: the thing that cost 43 of 68 segments on the 45 A set.
test_card_alignment_corroboration.py	8	A weak alignment peak that another card reproduces is a confirmed one.
test_card_diagnostics.py	5	Card diagnostics against the Gamry: per-card medians, the card-to-Gamry
test_card_gain.py	16	The per-card voltage-gain check and the DC closure, on data with a known answer.
test_channel_lag_stage.py	12	The channel-lag stage inside the pipeline, on a plate with a known answer.
test_console_encoding.py	6	The log must survive a console that cannot spell.
test_coverage_and_provenance.py	9	A frequency in the schedule is not a frequency that was measured.
test_csv_pipeline.py	5	Synthesises a plate whose per-segment impedance is known exactly, writes it out in each supported CSV layout, runs the pipeline over it and asks whether the numbers come back. Run it before trusting a real file: with Rs varying across the plate so the heat map
test_drt_nonnegativity.py	9	A distribution of relaxation times cannot be negative.
test_ecm_drt.py	7	ecm_drt: the revised DRT-informed ECM fit (method B).
test_famos_dialects.py	9	Two FAMOS dialects, and the one that fails silently.
test_figure_panels.py	9	One plot per figure.
test_file_discovery.py	7	Which card files belong to THIS run?
test_frequency_refine.py	2	Blind runs: the ladder snap puts every step on a perfect 10/decade grid,
test_frequency_response.py	6	frequency_response: card delays show before the timing correction and not
test_full_band.py	10	Full band (a developer diagnostic since the shipped evaluation stops at
test_gamry_compare.py	22	The whole-cell cross-check: does it recover an error we planted?
test_gamry_plotting.py	4	The Gamry is drawn exactly as recorded: every point of the .DTA at the
test_gamry_selection.py	14	Which cell's reference sweep is this, and how would you know?
test_gamry_sync.py	12	The Gamry clock: read its timeline, measure the offset, fix what the blind
test_heatmap_limits.py	3	Fixed heat-map colour scales (config.HEATMAP_LIMITS).
test_hf_response.py	4	The two fixes of the October 2026 multi-condition review:
test_hf_schedule.py	3	One stretch of the record is one step of the sweep.
test_hfr_estimators.py	8	How should the high-frequency intercept be measured?
test_plate_figure.py	5	plate_figure: the viewer-style plate map draws, and uses the fixed scale.
test_plate_maps.py	7	Plate maps with the value in every segment, on a scale that shows spread.
test_plate_stats.py	6	Plate mean / median over the 36 representative segments: segment 1..36
test_plate_style.py	13	One look for every plate map: Jet colours, value over number, the view
test_plausibility.py	32	Plausibility checks: do they catch the faults they claim to catch?
test_polcurve.py	5	polcurve: Gamry Vdc/HFR parsing, FAMOS current density, the plot.
test_prominence_guard.py	5	The prominence guard is a duration, and widening it costs nothing.
test_reference_channel.py	8	The shared reference is UC2 on every card, by name, not by argmax.
test_rejection_ledger.py	8	Why did a point, or a whole segment, not make it into the result?
test_runner_display_cells.py	7	What the display cells draw, and what they leave out.
test_runner_result_lookup.py	18	Which run do the display cells show?
test_runner_widgets_cell.py	14	The widgets cell, run cold.
test_segment_exclusion.py	12	Leaving a segment out has to mean leaving it out.
test_segment_scale.py	2	segment_scale: a constant per-segment factor is found, a real local
test_spectra_dir.py	8	The bench tool's spectra folders (Spectrum<n>_<T>degC_<I>A/z<n>.csv) end to

Test file	Tests	What it checks (module docstring)
test_standard_eis.py	5	The standard evaluation (standard_eis.py, config.param_method = "ecm"):
test_tone_estimation.py	5	Card-level tone estimation (tone_estimation.py) and its wiring into bronze.
test_via_resistance.py	3	via_resistance: the via coefficient's slope and temperature coefficient
test_viewers.py	5	viewers: one plot chosen from drop-downs; the plate viewer offers every
test_window_confine.py	5	Low-frequency windows must sit inside their own Gamry step.

7. Commands you will use

Command	What it does
`python run.py main --self-test`	geometry of both plates, estimators, CSV readers; no data needed
`python -m pytest tests -q`	the full test suite (~457 tests, ~5 min). Expected: 1 documented failure
`python pipeline/main.py --dat DIR --curr-cal curr.csv --temp-cal temp.csv --gamry GDIR --leepa 2612030 --condition 45A --out out/45A`	one full FAMOS run
`python run.py main --reevaluate RUN_DIR --out OUT`	silver + gold again from saved bronze CSVs
`python tests/make_synth_sweep.py /tmp/synth`	synthetic cards + Gamry file with known truth
`python run.py frequency_response RUN_DIR --bode ABGLEICH/bode`	chain response check
`python run.py dc_closure RUN_45A RUN_60A ... -o OUT`	plate vs bench current over conditions
`python readers/gamry_dta.py ABGLEICH/bode --curr-cal curr.csv -o chain_gain.csv`	build the chain gain file

7.1 Rules for changing the code

• **A new tunable number** goes into `core/config.py` as a `Config` field with a comment; never as a constant inside a module.

• **A new result-changing setting** must also be added to `_CACHE_IDENTITY_KEYS` in the runner, otherwise the cache serves results computed without it.

• **A new function a new notebook version depends on** goes into `_BUILD_MARKERS`, so a mixed-version folder is detected.

• **Areas** always through `utils.segment_areas(cfg)`; geometry always as `geom.SEGMENTS` after `geom.use_plate()`.

• **A dropped point** must get a ledger reason (use the `gate()` pattern in `silver.gate_points`).

• **A new check** must not be able to stop a run: wrap it in `try/except` and report a verdict.

• **Every fix gets a test** in `tests/` that plants the defect in synthetic data and asserts it is caught.

• After a code change, run the notebook once with **Run mode = rerun** (the cache key does not include the code version).

8. Glossary

Term	Meaning
ADC slot	position of a channel in the converter's scan order; multiplexed channels are sampled one after another
Abgleich	bench calibration delivery: curr.csv, temp.csv, bode sweeps, step files
Aggregate	segments in parallel: A_cell / Σ(A_s/Z_s), compared with the Gamry
ASR	area-specific resistance/impedance, Ω·cm²
Bronze / silver / gold	raw / corrected-and-modelled / plate-field layers
CFAR	constant false-alarm-rate detection test
CRLB	Cramér–Rao lower bound: the best possible estimator uncertainty
Dataclass	class whose __init__/__repr__/__eq__ are generated from annotated fields
datago	Bosch Delta tables holding the raw measurements
DRT	distribution of relaxation times
ECM / ZARC / CPE	equivalent-circuit model / R parallel to a constant-phase element
Fixture (pytest)	argument such as tmp_path or monkeypatch that pytest supplies to a test

Term	Meaning
gen1 / gen2	green Kashyyyk / blue Naboo plate
Ledger	point_rejections.csv: for each point, kept or which gate removed it
lin-KK / Z-HIT	spectrum validity tests (Kramers–Kronig; Hilbert transform)
Monkey-patching	replacing a module attribute at runtime (datago shim, pytest monkeypatch)
Phasor	complex amplitude of a sine at a known frequency
Tier A/B/C/D	segment quality grade (D = inferred, no own data)
UC2	the cell-voltage channel used as reference on every card